



Lesson 1: Kotlin basics

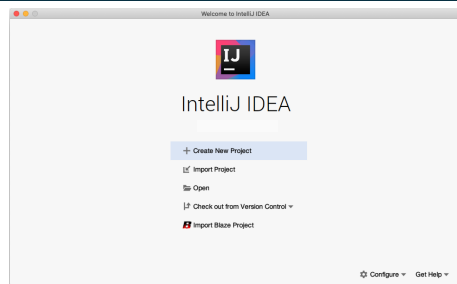
About this lesson

Lesson 1: Kotlin basics

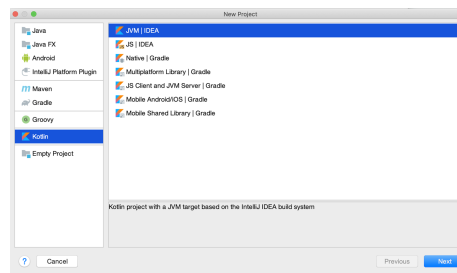
- [Get started](#)
- [Operators](#)
- [Data types](#)
- [Variables](#)
- [Conditionals](#)
- [Lists and arrays](#)
- [Null safety](#)
- [Summary](#)

Get started

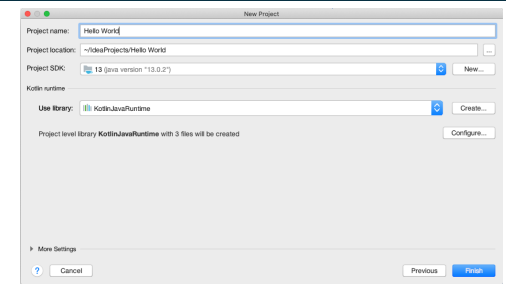
Open IntelliJ IDEA



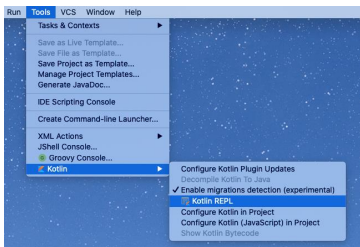
Create a new project



Name the project

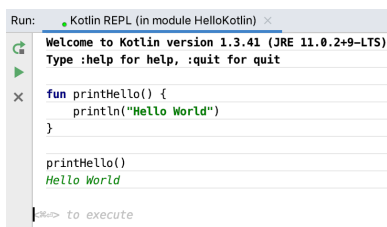


Open REPL (Read-Eval-Print-Loop)



It may take a few moments before the Kotlin menu appears under Tools.

Create a printHello() function



Press **Control+Enter** (Command+Enter on a Mac) to execute.

Operators

Operators

Math operators with integers

Math operators with doubles

- Mathematical operators
 - Increment and decrement operators
 - Comparison operators
 - Assignment operator
 - Equality operators

+ - * / %

++ --

< <= > >=

=

== !=
- 1 + 1 => 2

53 - 3 => 50

50 / 10 => 5

9 % 3 => 0
- 1.0 / 2.0 => 0.5

2.0 * 3.5 => 7.0

Math operators

1+1
=> kotlin.Int = 2

53-3
=> kotlin.Int = 50

50/10
=> kotlin.Int = 5

1.0/2.0
=> kotlin.Double = 0.5

2.0*3.5
=> kotlin.Double = 7.0

=> indicates output from your code.
Result includes the type (**kotlin.Int**).

Numeric operator methods

Kotlin keeps numbers as primitives, but lets you call methods on numbers as if they were objects.

2.times(3)
=> kotlin.Int = 6

3.5.plus(4)
=> kotlin.Double = 7.5

2.4.div(2)
=> kotlin.Double = 1.2

Data types

Integer types

Type	Bits	Notes
Long	64	From -2 ⁶³ to 2 ⁶³ -1
Int	32	From -2 ³¹ to 2 ³¹ -1
Short	16	From -32768 to 32767
Byte	8	From -128 to 127

Floating-point and other numeric types

Type	Bits	Notes
Double	64	16 - 17 significant digits
Float	32	6 - 7 significant digits
Char	16	16-bit Unicode character
Boolean	8	True or false. Operations include: - lazy disjunction, && - lazy conjunction, ! - negation

Operand types

Results of operations keep the types of the operands

6*50
=> kotlin.Int = 300

1/2
=> kotlin.Int = 0

6.0*50.0
=> kotlin.Double = 300.0

1.0*2.0
=> kotlin.Double = 0.5

6.0*50
=> kotlin.Double = 300.0

Type casting

Assign an `Int` to a `Byte`

```
val i: Int = 6
val b: Byte = i
println(b)

⇒ error: type mismatch: inferred type is Int but Byte was
expected
```

Convert `Int` to `Byte` with casting

```
val i: Int = 6
println(i.toByte())

⇒ 6
```

Underscores for long numbers

Use underscores to make long numeric constants more readable.

```
val oneMillion = 1_000_000
val idNumber = 999_99_9999L
val hexBytes = 0xFF_EC_DE_5E
val bytes = 0b11010010_01101001_10010100_10010010
```

Strings

Strings are any sequence of characters enclosed by double quotes.

```
val s1 = "Hello world!"
```

String literals can contain escape characters

```
val s2 = "Hello world!\n"
```

Or any arbitrary text delimited by a triple quote (" """)

```
val text = """
var bikes = 50
"""
```

String concatenation

```
val numberOfDogs = 3
val numberOfCats = 2
```

```
"I have $numberOfDogs dogs" + " and $numberOfCats cats"
```

```
⇒ I have 3 dogs and 2 cats
```

String templates

A template expression starts with a dollar sign (\$) and can be a simple value:

```
val i = 10
println("i = $i")

⇒ i = 10
```

Or an expression inside curly braces:

```
val s = "abc"
println("${s.length} is ${s.length}")

⇒ abc.length is 3
```

String template expressions

```
val numberOfShirts = 10
val numberOfPants = 5
```

```
"I have ${numberOfShirts + numberOfPants} items of clothing"
```

```
⇒ I have 15 items of clothing
```

Variables

Variables

- Powerful type inference
 - Let the compiler infer the type
 - You can explicitly declare the type if needed
- Mutable and immutable variables
 - Immutability not enforced, but recommended

Kotlin is a statically-typed language. The type is resolved at compile time and never changes.

Specifying the variable type

Colon Notation

```
var width: Int = 12
var length: Double = 2.5
```

Important: Once a type has been assigned by you or the compiler, you can't change the type or you get an error.

Mutable and immutable variables

- Mutable (Changeable)

```
var score = 10
```

- Immutable (Unchangeable)

```
val name = "Jennifer"
```

Although not strictly enforced, using immutable variables is recommended in most cases.

var and val

```
var count = 1
count = 2

val size = 1
size = 2

=> Error: val cannot be reassigned
```

Conditionals

Control flow

Kotlin features several ways to implement conditional logic:

- If/Else statements
- When statements
- For loops
- While loops

if/else statements

```
val numberOfCups = 30
val numberOfPlates = 50

if (numberOfCups > numberOfPlates) {
    println("Too many cups!")
} else {
    println("Not enough cups!")
}

=> Not enough cups!
```

if statement with multiple cases

```
val guests = 30
if (guests == 0) {
    println("No guests")
} else if (guests < 20) {
    println("Small group of people")
} else {
    println("Large group of people!")
}

=> Large group of people!
```

Ranges

- Data type containing a span of comparable values (e.g., integers from 1 to 100 inclusive)
- Ranges are bounded
- Objects within a range can be mutable or immutable

Ranges in if/else statements

```
val numberOfStudents = 50
if (numberOfStudents in 1..100) {
    println(numberOfStudents)
}

=> 50
```

Note: There are no spaces around the "range to" operator (1..100)

when statement

```
when (results) {
    0 -> println("No results")
    in 1..39 -> println("Got results!")
    else -> println("That's a lot of results!")
}

=> That's a lot of results!
```

As well as a when statement, you can also define a when expression that provides a return value.

for loops

```
val pets = arrayOf("dog", "cat", "canary")
for (element in pets) {
    print(element + " ")
}
⇒ dog cat canary
```

You don't need to define an iterator variable and increment it for each pass.

for loops: elements and indexes

```
for ((index, element) in pets.withIndex()) {
    println("Item at $index is $element\n")
}
⇒ Item at 0 is dog
Item at 1 is cat
Item at 2 is canary
```

for loops: step sizes and ranges

```
for (i in 1..5) print(i)
⇒ 12345

for (i in 5 downTo 1) print(i)
⇒ 54321

for (i in 3..6 step 2) print(i)
⇒ 35

for (i in 'd'..'g') print (i)
⇒ defg
```

while loops

```
var bicycles = 0
while (bicycles < 50) {
    bicycles++
}
println("$bicycles bicycles in the bicycle rack\n")
⇒ 50 bicycles in the bicycle rack

do {
    bicycles--
} while (bicycles > 50)
println("$bicycles bicycles in the bicycle rack\n")
⇒ 49 bicycles in the bicycle rack
```

repeat loops

```
repeat(2) {
    print("Hello!")
}
⇒ Hello!Hello!
```

Lists and arrays

Lists

- Lists are ordered collections of elements
- List elements can be accessed programmatically through their indices
- Elements can occur more than once in a list

An example of a list is a sentence: it's a group of words, their order is important, and they can repeat.

Immutable list using listOf()

Declare a list using `listOf()` and print it out.

```
val instruments = listOf("trumpet", "piano", "violin")
println(instruments)
⇒ [trumpet, piano, violin]
```

Mutable list using mutableListOf()

Lists can be changed using `mutableListOf()`

```
val myList = mutableListOf("trumpet", "piano", "violin")
myList.remove("violin")

⇒ kotlin.Boolean = true
```

With a list defined with `val`, you can't change which list the variable refers to, but you can still change the contents of the list.

Arrays

- Arrays store multiple items
- Array elements can be accessed programmatically through their indices
- Array elements are mutable
- Array size is fixed

Array using arrayOf()

An array of strings can be created using `arrayOf()`

```
val pets = arrayOf("dog", "cat", "canary")
println(java.util.Arrays.toString(pets))
⇒ [dog, cat, canary]
```

With an array defined with `val`, you can't change which array the variable refers to, but you can still change the contents of the array.

Arrays with mixed or single types

An array can contain different types.

```
val mix = arrayOf("hats", 2)
```

An array can also contain just one type (integers in this case).

```
val numbers = intArrayOf(1, 2, 3)
```

Combining arrays

Use the `+` operator.

```
val numbers = intArrayOf(1,2,3)
val numbers2 = intArrayOf(4,5,6)
val combined = numbers2 + numbers
println(Arrays.toString(combined))

⇒ [4, 5, 6, 1, 2, 3]
```

Null safety

Null safety

- In Kotlin, variables cannot be null by default
- You can explicitly assign a variable to null using the safe call operator
- Allow null-pointer exceptions using the `!!` operator
- You can test for null using the elvis (`? :`) operator

Variables cannot be null

In Kotlin, `null` variables are not allowed by default.

Declare an `Int` and assign `null` to it.

```
var numberOfBooks: Int = null
⇒ error: null can not be a value of a non-null type Int
```

Safe call operator

The safe call operator (`?`), after the type indicates that a variable can be `null`.

Declare an `Int?` as nullable

```
var numberOfBooks: Int? = null
```

In general, do not set a variable to null as it may have unwanted consequences.

Testing for null

Check whether the `numberOfBooks` variable is not `null`. Then decrement that variable.

```
var numberOfBooks = 6
if (numberOfBooks != null) {
    numberOfBooks = numberOfBooks.dec()
}
```

Now look at the Kotlin way of writing it, using the safe call operator.

```
var numberOfBooks = 6
numberOfBooks = numberOfBooks?.dec()
```

The !! operator

If you're certain a variable won't be null, use !! to force the variable into a non-null type. Then you can call methods/properties on it.

```
val len = s!!.length
```

throws NullPointerException if s is null

Warning: Because !! will throw an exception, it should only be used when it would be exceptional to hold a null value.

Elvis operator

Chain null tests with the ?: operator.

```
numberOfBooks = numberOfBooks?.dec() ?: 0
```

The ?: operator is sometimes called the "Elvis operator," because it's like a smiley on its side with a pompadour hairstyle, like Elvis Presley styled his hair.

Summary

Summary

In Lesson 1, you learned how to:

- Create an IntelliJ IDEA project, opening REPL, and execute a function
- Use operators and numeric operator methods
- Use data types, type casting, strings, and string templates
- Use variables and type inference, and mutable and immutable variables
- Use conditionals, control flow, and looping structures
- Use lists and arrays
- Use Kotlin's null safety features

Pathway

Practice what you've learned by completing the pathway:

[Lesson 1: Kotlin basics](#)



Summary



Lesson 2: Functions

About this lesson

Lesson 2: Functions

- [Programs in Kotlin](#)
- [\(Almost\) Everything has a value](#)
- [Functions in Kotlin](#)
- [Compact functions](#)
- [Lambdas and higher-order functions](#)
- [List filters](#)
- [Summary](#)

Programs in Kotlin

Setting up

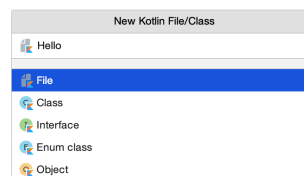
Before you can write code and run programs, you need to:

- Create a file in your project
- Create a `main()` function
- Pass arguments to `main()` (Optional)
- Use any passed arguments in function calls (Optional)
- Run your program

Create a new Kotlin file

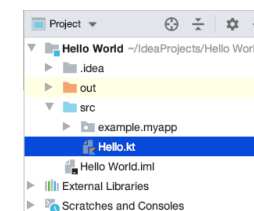
In IntelliJ IDEA's Project pane, under **Hello World**, right-click the `src` folder.

- Select **New > Kotlin File/Class**.
- Select **File**, name the file `Hello`, and press **Enter**.



Create a Kotlin file

You should now see a file in the `src` folder called `Hello.kt`.



Create a main() function

`main()` is the entry point for execution for a Kotlin program.

In the `Hello.kt` file:

```
fun main(args: Array<String>) {  
    println("Hello, world!")  
}
```

The args in the `main()` function are optional.

Run your Kotlin program

To run your program, click the Run icon (a green play button) to the left of the `main()` function.

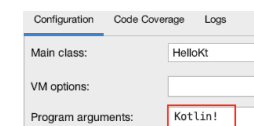
```
1 ▶ fun main(args: Array<String>) {  
2     println("Hello, world!")  
3 }  
4
```

IntelliJ IDEA runs the program, and displays the results in the console.

```
HelloKt  
/Library/Java/JavaVirtualMachines/jdk-13.0.2.jdk/Contents/Home/bin/java  
Hello, world!  
  
Process finished with exit code 0
```

Pass arguments to main()

Select **Run > Edit Configurations** to open the **Run/Debug Configurations** window.



Use arguments in main()

Use `args[0]` to access the first input argument passed to `main()`.

```
fun main(args: Array<String>) {  
    println("Hello, ${args[0]}")  
}  
  
⇒ Hello, Kotlin!
```

(Almost) Everything has a value

(Almost) Everything is an expression

In Kotlin, almost everything is an expression and has a value. Even an `if` expression has a value.

```
val temperature = 20  
val isHot = if (temperature > 40) true else false  
println(isHot)  
  
⇒ false
```

Expression values

Sometimes, that value is `kotlin.Unit`.

```
val isUnit = println("This is an expression")  
println(isUnit)  
  
⇒ This is an expression  
   kotlin.Unit
```

Functions in Kotlin

About functions

- A block of code that performs a specific task
- Breaks a large program into smaller modular chunks
- Declared using the `fun` keyword
- Can take arguments with either named or default values

Parts of a function

Earlier, you created a simple function that printed "Hello World".

```
fun printHello() {  
    println("Hello World")  
}  
  
printHello()
```

Unit returning functions

If a function does not return any useful value, its return type is `Unit`.

```
fun printHello(name: String?): Unit {  
    println("Hi there!")  
}
```

`Unit` is a type with only one value: `Unit`.

Unit returning functions

The `Unit` return type declaration is optional.

```
fun printHello(name: String?): Unit {  
    println("Hi there!")  
}
```

is equivalent to:

```
fun printHello(name: String?) {  
    println("Hi there!")  
}
```

Function arguments

Functions may have:

- Default parameters
- Required parameters
- Named arguments

Default parameters

Default values provide a fallback if no parameter value is passed.

```
fun drive(speed: String = "fast") {  
    println("driving $speed")  
}
```

Use "=" after the type to define default values

drive() ⇒ driving fast
drive("slow") ⇒ driving slow
drive(speed = "turtle-like") ⇒ driving turtle-like

Required parameters

If no default is specified for a parameter, the corresponding argument is required.

Required parameters

```
fun tempToday(day: String, temp: Int) {  
    println("Today is $day and it's $temp degrees.")  
}
```

Default versus required parameters

Functions can have a mix of default and required parameters.

```
fun reformat(str: String,  
            divideByCamelHumps: Boolean,  
            wordSeparator: Char,  
            normalizeCase: Boolean = true){
```

Has default value

Pass in required arguments.

```
reformat("Today is a day like no other day", false, '_')
```

Named arguments

To improve readability, use named arguments for required arguments.

```
reformat(str, divideByCamelHumps = false, wordSeparator = '_')
```

It's considered good style to put default arguments after positional arguments, that way callers only have to specify the required arguments.

Compact functions

Single-expression functions

Compact functions, or single-expression functions, make your code more concise and readable.

```
fun double(x: Int): Int {  
    x * 2  
}
```

Complete version

```
fun double(x: Int): Int = x * 2
```

Compact version

Lambdas and higher-order functions

Kotlin functions are first-class

- Kotlin functions can be stored in variables and data structures
- They can be passed as arguments to, and returned from, other higher-order functions
- You can use higher-order functions to create new "built-in" functions

Lambda functions

A lambda is an expression that makes a function that has no name.

```
var dirtLevel = 20
val waterFilter = {level: Int -> level / 2}
println(waterFilter(dirtLevel))
⇒ 10
```

Parameter and type
Function arrow
Code to execute

Syntax for function types

Kotlin's syntax for function types is closely related to its syntax for lambdas. Declare a variable that holds a function.

```
val waterFilter: (Int) -> Int = {level -> level / 2}
```

Variable name Data type of variable (function type) Function

Higher-order functions

Higher-order functions take functions as parameters, or return a function.

```
fun encodeMsg(msg: String, encode: (String) -> String): String {
    return encode(msg)
}
```

The body of the code calls the function that was passed as the second argument, and passes the first argument along to it.

Higher-order functions

To call this function, pass it a string and a function.

```
val enc1: (String) -> String = { input -> input.toUpperCase() }
println(encodeMsg("abc", enc1))
```

Using a function type separates its implementation from its usage.

Passing a function reference

Use the :: operator to pass a named function as an argument to another function.

```
fun enc2(input:String): String = input.reversed()
encodeMessage("abc", ::enc2)
```

Passing a named function, not a lambda

The :: operator lets Kotlin know that you are passing the function reference as an argument, and not trying to call the function.

Last parameter call syntax

Kotlin prefers that any parameter that takes a function is the last parameter.

```
encodeMessage("acronym", { input -> input.toUpperCase() })
```

You can pass a lambda as a function parameter without putting it inside the parentheses.

```
encodeMsg("acronym") { input -> input.toUpperCase() }
```

Using higher-order functions

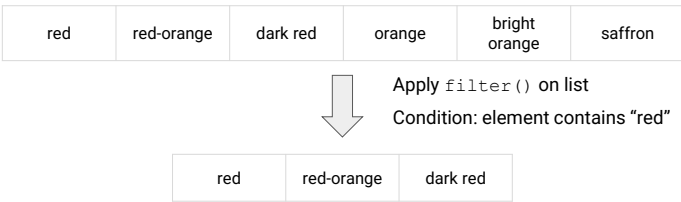
Many Kotlin built-in functions are defined using last parameter call syntax.

```
inline fun repeat(times: Int, action: (Int) -> Unit)
repeat(3) {
    println("Hello")
}
```

List filters

List filters

Get part of a list based on some condition



Iterating through lists

If a function literal has only one parameter, you can omit its declaration and the "->". The parameter is implicitly declared under the name `it`.

```
val ints = listOf(1, 2, 3)
ints.filter { it > 0 }
```

Filter iterates through a collection, where `it` is the value of the element during the iteration. This is equivalent to:

```
ints.filter { n: Int -> n > 0 } OR ints.filter { n -> n > 0 }
```

List filters

The filter condition in curly braces `{ }` tests each item as the filter loops through. If the expression returns `true`, the item is included.

```
val books = listOf("nature", "biology", "birds")
println(books.filter { it[0] == 'b' })
=> [biology, birds]
```

Eager and lazy filters

Evaluation of expressions in lists:

- **Eager:** occurs regardless of whether the result is ever used
- **Lazy:** occurs only if necessary at runtime

Lazy evaluation of lists is useful if you don't need the entire result, or if the list is exceptionally large and multiple copies wouldn't fit into RAM.

Eager filters

Filters are eager by default. A new list is created each time you use a filter.

```
val instruments = listOf("viola", "cello", "violin")
val eager = instruments.filter { it [0] == 'v' }
println("eager: " + eager)
=> eager: [viola, violin]
```

Lazy filters

Sequences are data structures that use lazy evaluation, and can be used with filters to make them lazy.

```
val instruments = listOf("viola", "cello", "violin")
val filtered = instruments.asSequence().filter { it[0] == 'v' }
println("filtered: " + filtered)
=> filtered: kotlin.sequences.FilteringSequence@386cc1c4
```

Sequences -> lists

Sequences can be turned back into lists using `toList()`.

```
val filtered = instruments.asSequence().filter { it[0] == 'v' }
val newList = filtered.toList()
println("new list: " + newList)
=> new list: [viola, violin]
```

Other list transformations

- `map()` performs the same transform on every item and returns the list.

```
val numbers = setOf(1, 2, 3)
println(numbers.map { it * 3 })
=> [3, 6, 9]
```

- `flatten()` returns a single list of all the elements of nested collections.

```
val numberSets = listOf(setOf(1, 2, 3), setOf(4, 5), setOf(1, 2))
println(numberSets.flatten())
=> [1, 2, 3, 4, 5, 1, 2]
```

Summary

Summary

In Lesson 2, you learned how to:

- Create a file and a `main()` function in your project, and run a program
- Pass arguments to the `main()` function
- Use the returned value of an expression
- Use default arguments to replace multiple versions of a function
- Use compact functions, to make code more readable
- Use lambdas and higher-order functions
- Use eager and lazy list filters

Pathway

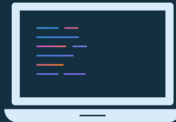
Practice what you've learned by completing the pathway:

[Lesson 2: Functions](#)





Lesson 3: Classes and objects



About this lesson

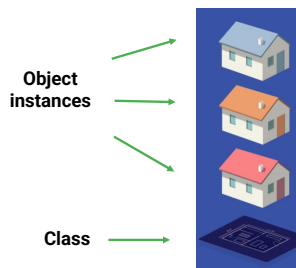
Lesson 3: Classes and objects

- [Classes](#)
- [Inheritance](#)
- [Extension functions](#)
- [Special classes](#)
- [Organizing your code](#)
- [Summary](#)

Classes

Class

- Classes are blueprints for objects
- Classes define methods that operate on their object instances



Class versus object instance

House Class

Data

- House color (String)
- Number of windows (Int)
- Is for sale (Boolean)

Behavior

- `updateColor()`
- `putOnSale()`

Object Instances



Define and use a class

Class Definition

```
class House {  
    val color: String = "white"  
    val numberOfWindows: Int = 2  
    val isForSale: Boolean = false  
  
    fun updateColor(newColor: String){...}  
    ...  
}
```

Create New Object Instance

```
val myHouse = House()  
println(myHouse)
```

Constructors

When a constructor is defined in the class header, it can contain:

- No parameters

```
class A
```
- Parameters
 - Not marked with `var` or `val` → copy exists only within scope of the constructor

```
class B(x: Int)
```
 - Marked `var` or `val` → copy exists in all instances of the class

```
class C(val y: Int)
```

Constructor examples

```
class A
```

```
val aa = A()
```

```
class B(x: Int)
```

```
val bb = B(12)  
println(bb.x)  
=> compiler error unresolved  
reference
```

```
class C(val y: Int)
```

```
val cc = C(42)  
println(cc.y)  
=> 42
```

Default parameters

Class instances can have default values.

- Use default values to reduce the number of constructors needed
- Default parameters can be mixed with required parameters
- More concise (don't need to have multiple constructor versions)

```
class Box(val length: Int, val width: Int = 20, val height: Int = 40)  
val box1 = Box(100, 20, 40)  
val box2 = Box(length = 100)  
val box3 = Box(length = 100, width = 20, height = 40)
```

Primary constructor

Declare the primary constructor within the class header.

```
class Circle(i: Int) {  
    init {  
        ...  
    }  
}
```

This is technically equivalent to:

```
class Circle {  
    constructor(i: Int) {  
        ...  
    }  
}
```

Initializer block

- Any required initialization code is run in a special `init` block
- Multiple `init` blocks are allowed
- `init` blocks become the body of the primary constructor

Initializer block example

Use the `init` keyword:

```
class Square(val side: Int) {  
    init {  
        println(side * 2)  
    }  
}  
  
val s = Square(10)  
=> 20
```

Multiple constructors

- Use the `constructor` keyword to define secondary constructors
- Secondary constructors must call:
 - The primary constructor using `this` keyword
 - OR
 - Another secondary constructor that calls the primary constructor
- Secondary constructor body is not required

Multiple constructors example

```
class Circle(val radius: Double) {  
    constructor(name: String) : this(1.0)  
    constructor(diameter: Int) : this(diameter / 2.0) {  
        println("in diameter constructor")  
    }  
    init {  
        println("Area: ${Math.PI * radius * radius}")  
    }  
}  
  
val c = Circle(3)
```

Properties

- Define properties in a class using `val` or `var`
- Access these properties using dot `.` notation with property name
- Set these properties using dot `.` notation with property name (only if declared a `var`)

Person class with name property

```
class Person(var name: String)  
  
fun main() {  
    val person = Person("Alex")  
  
    println(person.name) ← Access with .<property name>  
    person.name = "Joey" ← Set with .<property name>  
    println(person.name)  
}
```

Custom getters and setters

If you don't want the default `get/set` behavior:

- Override `get()` for a property
- Override `set()` for a property (if defined as a `var`)

Format: `var propertyName: DataType = initialValue`

```
    get() = ...  
    set(value) {  
        ...  
    }
```

Custom getter

```
class Person(val firstName: String, val lastName: String) {  
    val fullName: String  
        get() {  
            return "$firstName $lastName"  
        }  
}  
  
val person = Person("John", "Doe")  
println(person.fullName)  
=> John Doe
```

Custom setter

```
var fullName:String = ""
get() = "$firstName $lastName"
set(value) {
    val components = value.split(" ")
    firstName = components[0]
    lastName = components[1]
    field = value
}

person.fullName = "Jane Smith"
```

Member functions

- Classes can also contain functions
- Declare functions as shown in *Functions* in Lesson 2
 - `fun` keyword
 - Can have default or required parameters
 - Specify return type (if not `Unit`)

Inheritance

Inheritance

- Kotlin has single-parent class inheritance
- Each class has exactly one parent class, called a superclass
- Each subclass inherits all members of its superclass including ones that the superclass itself has inherited

If you don't want to be limited by only inheriting a single class, you can define an interface since you can implement as many of those as you want.

Interfaces

- Provide a contract all implementing classes must adhere to
- Can contain method signatures and property names
- Can derive from other interfaces

Format: `interface` NameOfInterface { interfaceBody }

Interface example

```
interface Shape {
    fun computeArea() : Double
}

class Circle(val radius:Double) : Shape {
    override fun computeArea() = Math.PI * radius * radius
}

val c = Circle(3.0)
println(c.computeArea())
=> 28.274333882308138
```

Extending classes

To extend a class:

- Create a new class that uses an existing class as its core (subclass)
- Add functionality to a class without creating a new one (extension functions)

Creating a new class

- Kotlin classes by default are not subclassable
- Use `open` keyword to allow subclassing
- Properties and functions are redefined with the `override` keyword

Classes are final by default

Declare a class

```
class A
```

Try to subclass A

```
class B : A
```

=>Error: A is final and cannot be inherited from

Use open keyword

Use `open` to declare a class so that it can be subclassed.

Declare a class

```
open class C
```

Subclass from C

```
class D : C()
```

Overriding

- Must use `open` for properties and methods that can be overridden (otherwise you get compiler error)
- Must use `override` when overriding properties and methods
- Something marked `override` can be overridden in subclasses (unless marked `final`)

Abstract classes

- Class is marked as `abstract`
- Cannot be instantiated, must be subclassed
- Similar to an interface with the added the ability to store state
- Properties and functions marked with `abstract` must be overridden
- Can include non-abstract properties and functions

Example abstract classes

```
abstract class Food {
    abstract val kcal : Int
    abstract val name : String
    fun consume() = println("I'm eating ${name}")
}
class Pizza() : Food() {
    override val kcal = 600
    override val name = "Pizza"
}
fun main() {
    Pizza().consume()    // "I'm eating Pizza"
}
```

When to use each

- Defining a broad spectrum of behavior or types? Consider an interface.
- Will the behavior be specific to that type? Consider a class.
- Need to inherit from multiple classes? Consider refactoring code to see if some behavior can be isolated into an interface.
- Want to leave some properties / methods abstract to be defined by subclasses? Consider an abstract class.
- You can extend only one class, but implement one or more interfaces.

Extension functions

Extension functions

Add functions to an existing class that you cannot modify directly.

- Appears as if the implementer added it
- Not actually modifying the existing class
- Cannot access private instance variables

Format: `fun` `ClassName`.`functionName`(`params`) { `body` }

Why use extension functions?

- Add functionality to classes that are not open
- Add functionality to classes you don't own
- Separate out core API from helper methods for classes you own

Define extension functions in an easily discoverable place such as in the same file as the class, or a well-named function.

Extension function example

Add `isOdd()` to `Int` class:

```
fun Int.isOdd(): Boolean { return this % 2 == 1 }
```

Call `isOdd()` on an `Int`:

```
3.isOdd()
```

Extension functions are very powerful in Kotlin!

Special classes

Data class

- Special class that exists just to store a set of data
- Mark the class with the `data` keyword
- Generates getters for each property (and setters for vars too)
- Generates `toString()`, `equals()`, `hashCode()`, `copy()` methods, and destructuring operators

Format: `data class` <NameOfClass>(parameterList)

Data class example

Define the data class:

```
data class Player(val name: String, val score: Int)
```

Use the data class:

```
val firstPlayer = Player("Lauren", 10)
println(firstPlayer)
=> Player(name=Lauren, score=10)
```

Data classes make your code much more concise!

Pair and Triple

- `Pair` and `Triple` are predefined data classes that store 2 or 3 pieces of data respectively
- Access variables with `.first`, `.second`, `.third` respectively
- Usually named data classes are a better option (more meaningful names for your use case)

Pair and Triple examples

```
val bookAuthor = Pair("Harry Potter", "J.K. Rowling")
println(bookAuthor)
=> (Harry Potter, J.K. Rowling)
```

```
val bookAuthorYear = Triple("Harry Potter", "J.K. Rowling", 1997)
println(bookAuthorYear)
println(bookAuthorYear.third)
=> (Harry Potter, J.K. Rowling, 1997)
1997
```

Pair to

`Pair`'s special `to` variant lets you omit parentheses and periods (infix function).

It allows for more readable code

```
val bookAuth1 = "Harry Potter".to("J. K. Rowling")
val bookAuth2 = "Harry Potter" to "J. K. Rowling"
=> bookAuth1 and bookAuth2 are Pair (Harry Potter, J. K. Rowling)
```

Also used in collections like `Map` and `HashMap`

```
val map = mapOf(1 to "x", 2 to "y", 3 to "zz")
=> map of Int to String {1=x, 2=y, 3=zz}
```

Enum class

User-defined data type for a set of named values

- Use `this` to require instances be one of several constant values
- The constant value is, by default, not visible to you
- Use `enum` before the `class` keyword

Format: `enum class` EnumName { NAME1, NAME2, ... NAMEn }

Referenced via `EnumName.<ConstantName>`

Enum class example

Define an `enum` with red, green, and blue colors.

```
enum class Color(val r: Int, val g: Int, val b: Int) {
    RED(255, 0, 0), GREEN(0, 255, 0), BLUE(0, 0, 255)
}

println(" " + Color.RED.r + " " + Color.RED.g + " " + Color.RED.b)
=> 255 0 0
```

Object/singleton

- Sometimes you only want one instance of a class to ever exist
- Use the `object` keyword instead of the `class` keyword
- Accessed with `NameOfObject.<function or variable>`

Object/singleton example

```
object Calculator {  
    fun add(n1: Int, n2: Int): Int {  
        return n1 + n2  
    }  
}  
  
println(Calculator.add(2,4))  
=> 6
```

Companion objects

- Lets all instances of a class share a single instance of a set of variables or functions
- Use `companion` keyword
- Referenced via `ClassName.PropertyOrFunction`

Companion object example

```
class PhysicsSystem {  
    companion object WorldConstants {  
        val gravity = 9.8  
        val unit = "metric"  
        fun computeForce(mass: Double, accel: Double): Double {  
            return mass * accel  
        }  
    }  
}  
  
println(PhysicsSystem.WorldConstants.gravity)  
println(PhysicsSystem.WorldConstants.computeForce(10.0, 10.0))  
  
=> 9.8100.0
```

Organizing your code

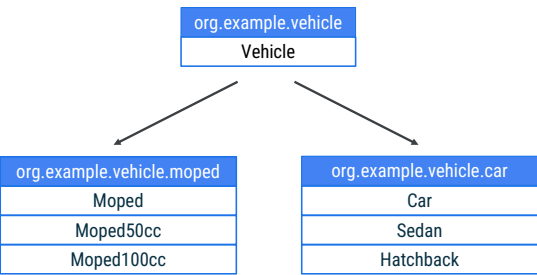
Single file, multiple entities

- Kotlin DOES NOT enforce a single entity (class/interface) per file convention
- You can and should group related structures in the same file
- Be mindful of file length and clutter

Packages

- Provide means for organization
 - Identifiers are generally lower case words separated by periods
 - Declared in the first non-comment line of code in a file following the `package` keyword
- ```
package org.example.game
```

# Example class hierarchy



# Visibility modifiers

Use visibility modifiers to limit what information you expose.

- `public` means visible outside the class. Everything is public by default, including variables and methods of the class.
- `private` means it will only be visible in that class (or source file if you are working with functions).
- `protected` is the same as `private`, but it will also be visible to any subclasses.

# Summary

## Summary

## Pathway

In Lesson 3, you learned about:

- Classes, constructors, and getters and setters
- Inheritance, interfaces, and how to extend classes
- Extension functions
- Special classes: data classes, enums, object/singletons, companion objects
- Packages
- Visibility modifiers

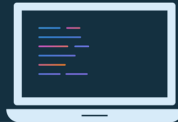
Practice what you've learned by completing the pathway:

[Lesson 3: Classes and objects](#)





## Lesson 4: Build your first Android app



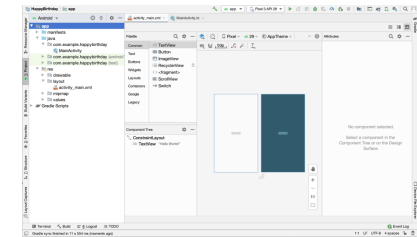
## About this lesson

### Lesson 4: Build your first Android app

- [Your first app](#)
- [Anatomy of an Android app](#)
- [Layouts and resources in Android](#)
- [Activities](#)
- [Make an app interactive](#)
- [Gradle: Building an Android app](#)
- [Accessibility](#)
- [Summary](#)

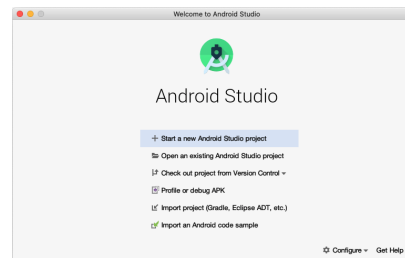
## Android Studio

Official IDE for building Android apps

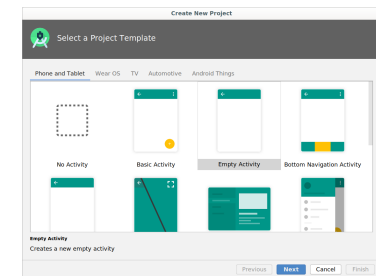


## Your first app

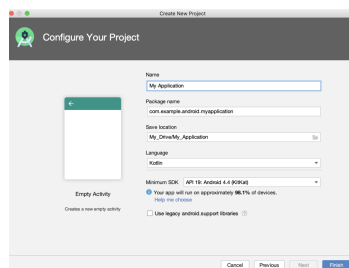
## Open Android Studio



## Create new project



## Enter your project details



## Android releases and API levels

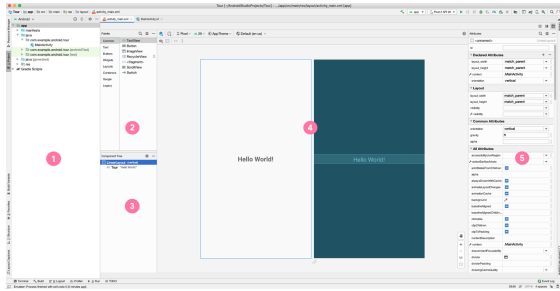
| Platform Version | API Level | VERSION_CODE |
|------------------|-----------|--------------|
| Android 10.0     | 29        | Q            |
| Android 9        | 28        | P            |
| Android 8.1      | 27        | O_MR1        |
| Android 8.0      | 26        | O            |
| Android 7.1.1    | 25        | N_MR1        |
| Android 7.1      |           |              |
| Android 7.0      | 24        | N            |
| Android 6.0      | 23        | M            |
| Android 5.1      | 22        | LOLLIPOP_MR1 |
| Android 5.0      | 21        | LOLLIPOP     |

## Choose API levels for your app

- Minimum SDK: Device needs at least this API level to install
- Target SDK: API version and highest Android version tested
- Compile SDK: Android OS library version compiled with  
`minSdkVersion <= targetSdkVersion <= compileSdkVersion`

The API level identifies the framework API version of the Android SDK.

## Tour of Android Studio



## Run your app



- Android device (phone, tablet)
- Emulator on your computer

## Android Virtual Device (AVD) Manager



## Anatomy of an Android App project

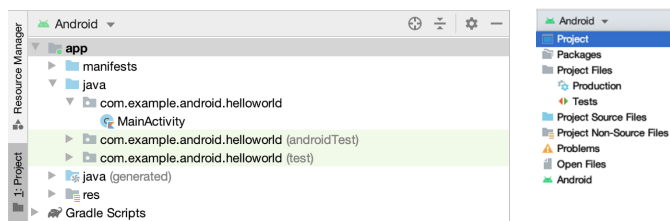
### Anatomy of a basic app project

- Activity
- Resources (layout files, images, audio files, themes, and colors)
- Gradle files

### Android app project structure

```
MyApplication
├── app
│ ├── libs
│ └── src
│ ├── androidTest
│ ├── main
│ │ ├── java
│ │ ├── res
│ │ └── AndroidManifest.xml
│ └── test
├── build.gradle
└── gradlew
```

## Browse files in Android Studio

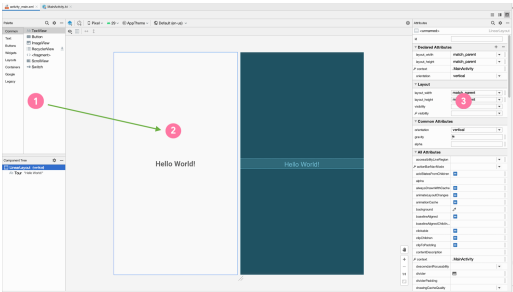


## Layouts and resources in Android

## Views

- Views are the user interface building blocks in Android
  - Bounded by a rectangular area on the screen
  - Responsible for drawing and event handling
  - Examples: TextView, ImageView, Button
- Can be grouped to form more complex user interfaces

# Layout Editor



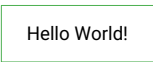
# XML Layouts

You can also edit your layout in XML.

- Android uses XML to specify the layout of user interfaces (including View attributes)
- Each View in XML corresponds to a class in Kotlin that controls how that View functions

# XML for a TextView

```
<TextView
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello World!"/>
```

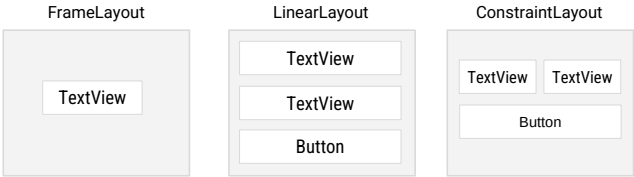


# Size of a View

- wrap\_content  
android:layout\_width="wrap\_content"
- match\_parent  
android:layout\_width="match\_parent"
- Fixed value (use dp units)  
android:layout\_width="48dp"

# ViewGroups

A ViewGroup is a container that determines how views are displayed.

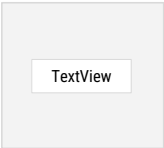


The ViewGroup is the parent and the views inside it are its children.

# FrameLayout example

A `FrameLayout` generally holds a single child View.

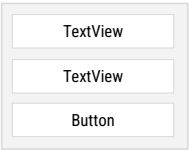
```
<FrameLayout
 android:layout_width="match_parent"
 android:layout_height="match_parent">
 <TextView
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:text="Hello World!"/>
</FrameLayout>
```



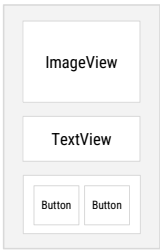
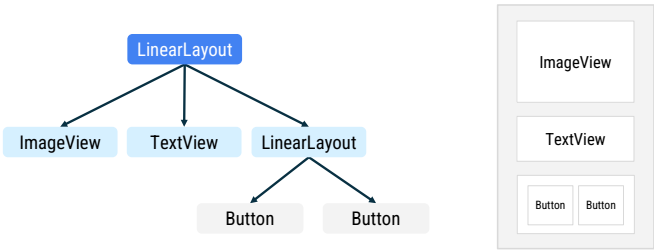
# LinearLayout example

- Aligns child views in a row or column
- Set `android:orientation` to horizontal or vertical

```
<LinearLayout
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="vertical">
 <TextView ... />
 <TextView ... />
 <Button ... />
</LinearLayout>
```



# View hierarchy



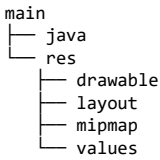
# App resources

Static content or additional files that your code uses

- Layout files
- Images
- Audio files
- User interface strings
- App icon

# Common resource directories

Add resources to your app by including them in the appropriate resource directory under the parent `res` folder.



# Resource IDs

- Each resource has a resource ID to access it.
  - When naming resources, the convention is to use all lowercase with underscores (for example, `activity_main.xml`).
  - Android autogenerated a class file named `R.java` with references to all resources in the app.
  - Individual items are referenced with:  
`R.<resource_type>.<resource_name>`
- Examples: `R.drawable.ic_launcher` (`res/drawable/ic_launcher.xml`)  
`R.layout.activity_main` (`res/layout/activity_main.xml`)

# Resource IDs for views

Individual views can also have resource IDs.

Add the `android:id` attribute to the View in XML. Use `@+id/name` syntax.

```
<TextView
 android:id="@+id/helloTextView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello World!"/>
```

Within your app, you can now refer to this specific TextView using:  
`R.id.helloTextView`

# Activities

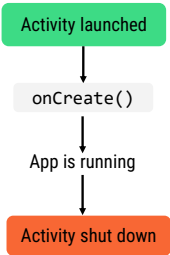
## What's an Activity?

- An Activity is a means for the user to accomplish one main goal.
  - An Android app is composed of one or more activities.
- 

## MainActivity.kt

```
class MainActivity : AppCompatActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)
 }
}
```

# How an Activity runs

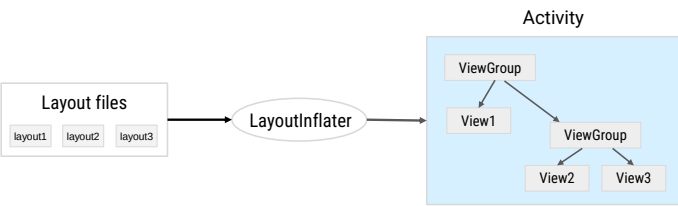


# Implement the onCreate() callback

Called when the system creates your Activity

```
override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)
}
```

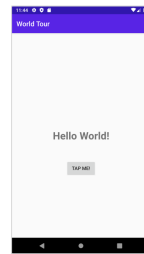
# Layout inflation



# Make an app interactive

## Define app behavior in Activity

Modify the Activity so the app responds to user input, such as a button tap.



## Modify a View dynamically

Within MainActivity.kt:

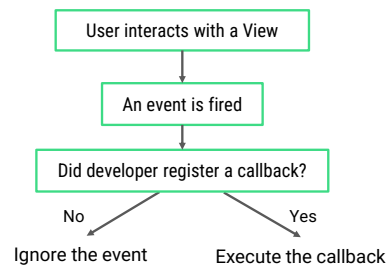
Get a reference to the View in the view hierarchy:

```
val resultTextView: TextView = findViewById(R.id.textview)
```

Change properties or call methods on the View instance:

```
resultTextView.text = "Goodbye!"
```

## Set up listeners for specific events



## View.OnClickListener

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
 override fun onCreate(savedInstanceState: Bundle?) {
 ...
 val button: Button = findViewById(R.id.button)
 button.setOnClickListener(this)
 }
 override fun onClick(v: View?) {
 TODO("not implemented")
 }
}
```

## SAM (single abstract method)

Converts a function into an implementation of an interface

**Format:** InterfaceName { lambda body }

```
val runnable = Runnable { println("Hi there") }
```

is equivalent to

```
val runnable = (object: Runnable {
 override fun run() {
 println("Hi there")
 }
})
```

## View.OnClickListener as a SAM

A more concise way to declare a click listener

```
class MainActivity : AppCompatActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 ...
 val button: Button = findViewById(R.id.button)
 button.setOnClickListener({ view -> /* do something*/ })
 }
}
```

## Late initialization

```
class Student(val id: String) {
 lateinit var records: HashSet<Any>
 init {
 // retrieve records given an id
 }
}
```

## Lateinit example in Activity

```
class MainActivity : AppCompatActivity() {
 lateinit var result: TextView
 override fun onCreate(savedInstanceState: Bundle?) {
 ...
 result = findViewById(R.id.result_text_view)
 }
}
```

# Gradle: Building an Android app

## What is Gradle?

- Builds automation system
- Manages the build cycle via a series of tasks (for example, compiles Kotlin sources, runs tests, installs app to device)
- Determines the proper order of tasks to run
- Manages dependencies between projects and third-party libraries

## Gradle build file

- Declare plugins
- Define Android properties
- Handle dependencies
- Connect to repositories

## Plugins

Provide libraries and infrastructure needed by your app

```
apply plugin: 'com.android.application'

apply plugin: 'kotlin-android'

apply plugin: 'kotlin-android-extensions'
```

## Android configuration

```
android {
 compileSdkVersion 30
 buildToolsVersion "30.0.2"

 defaultConfig {
 applicationId "com.example.sample"
 minSdkVersion 19
 targetSdkVersion 30
 }
}
```

## Dependencies

```
dependencies {
 implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
 implementation 'androidx.core:core-ktx:1.3.2'
 implementation 'androidx.appcompat:appcompat:1.2.0'
 implementation 'com.google.android.material:material:1.2.1'
 ...
}
```

## Repositories

```
repositories {
 google()
 mavenCentral()
}
```

## Common Gradle tasks

- Clean
- Tasks
- InstallDebug

# Accessibility

## Accessibility

- Refers to improving the design and functionality of your app to make it easier for more people, including those with disabilities, to use
- Making your app more accessible leads to an overall better user experience and benefits all your users

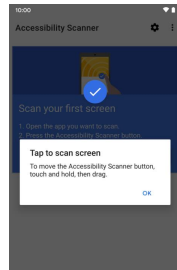
## Make apps more accessible

- Increase text visibility with foreground and background color contrast ratio:
  - At least 4.5:1 for small text against the background
  - At least 3.0:1 for large text against the background
- Use large, simple controls
  - Touch target size should be at least 48dp x 48dp
- Describe each UI element
  - Set content description on images and controls

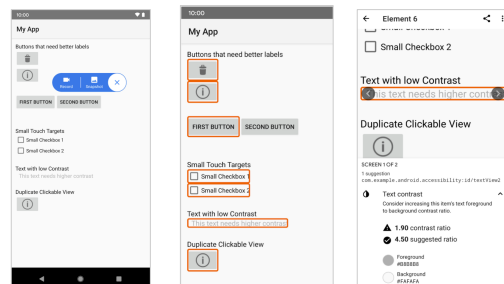
## Accessibility Scanner

Tool that scans your screen and suggests improvements to make your app more accessible, based on:

- Content labels
- Touch target sizes
- Clickable views
- Text and image contrast



## Accessibility Scanner example



## Add content labels

- Set `contentDescription` attribute → read aloud by screen reader
- ```
<ImageView  
...  
    android:contentDescription="@string/stop_sign" />
```
- Text in `TextView` already provided to accessibility services, no additional label needed

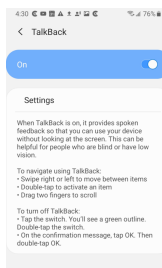
No content label needed

- For graphical elements that are purely for decorative purposes, you can set `android:importantForAccessibility="no"`
- Removing unnecessary announcements is better for the user

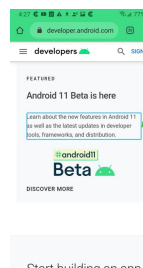
TalkBack

- Google screen reader included on Android devices
- Provides spoken feedback so you don't have to look at the screen to use your device
- Lets you navigate the device using gestures
- Includes braille keyboard for Unified English Braille

TalkBack example



Reads text aloud as user navigates the screen



Switch access

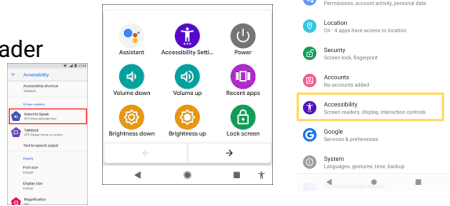
- Allows for controlling the device using one or more switches instead of the touchscreen
- Switch Access will let users move through menus, home screen icons, or other items and select the appropriate action
- Scans your app UI and highlights each item until you make a selection
- Use with external switch, external keyboard, or buttons on the Android device (e.g., volume buttons)



Android Accessibility Suite

Collection of accessibility apps that help you use your Android device eyes-free, or with a switch device. It includes:

- Talkback screen reader
- Switch Access
- Accessibility Menu
- Select to Speak



Accessibility Resources

- [Build more accessible apps](#)
- [Principles for improving app accessibility](#)
- [Basic Android Accessibility codelab](#)
- [Material Design best practices on accessibility](#)

Summary

Summary

In Lesson 4, you learned how to:

- Use Views and ViewGroups to build the user interface of your app
- Access resources in your app from `R.<resource_type>.<resource_name>`
- Define app behavior in the Activity (for example, register `OnClickListener`)
- Use Gradle as the build system to build your app
- Follow best practices to make your apps more accessible

Learn more

- [Layouts](#)
- [LinearLayout](#)
- [Input events overview](#)
- [View](#)
- [ViewGroup](#)



Lesson 5: Layouts

About this lesson

Lesson 5: Layouts

- [Layouts in Android](#)
- [ConstraintLayout](#)
- [Additional topics for ConstraintLayout](#)
- [Data binding](#)
- [Displaying lists with RecyclerView](#)
- [Summary](#)

Layouts in Android

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

ConstraintLayout

Deeply nested layouts are costly

- Deeply nested ViewGroups require more computation
- Views may be measured multiple times
- Can cause UI slowdown and lack of responsiveness

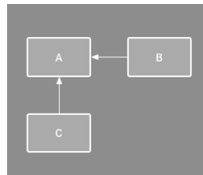
Use ConstraintLayout to avoid some of these issues!

What is ConstraintLayout?

- Recommended default layout for Android
- Solves costly issue of too many nested layouts, while allowing complex behavior
- Position and size views within it using a set of constraints

What is a constraint?

A restriction or limitation on the properties of a View that the layout attempts to respect



Relative positioning constraints

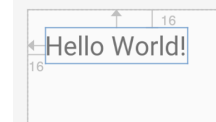
Can set up a constraint relative to the parent container

Format:

`layout_constraint<SourceConstraint>_to<TargetConstraint>Of`

Example attributes on a TextView:

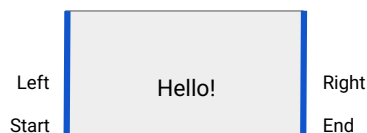
```
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
```



Relative positioning constraints



Relative positioning constraints



Simple ConstraintLayout example

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

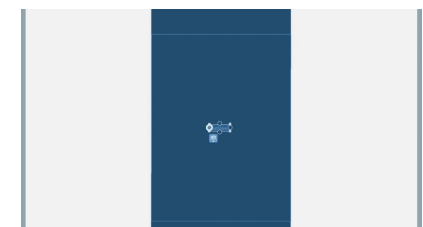
    <TextView
        ...
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```



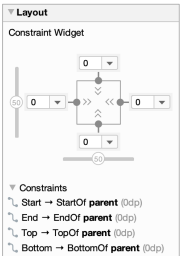
Layout Editor in Android Studio

You can click and drag to add constraints to a View.

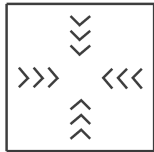
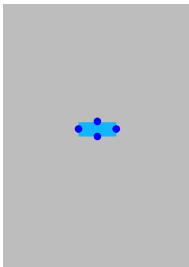


Constraint Widget in Layout Editor

- Fixed
- Wrap content
- Match constraints

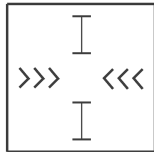
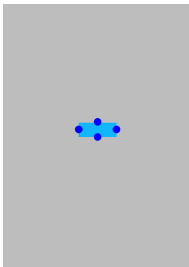


Wrap content for width and height



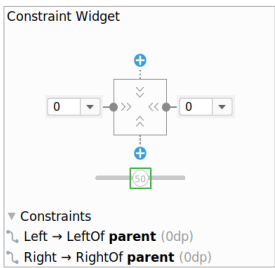
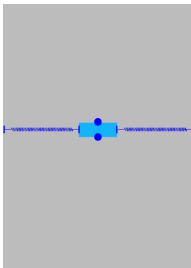
layout_width wrap_content
layout_height wrap_content

Wrap content for width, fixed height



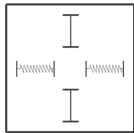
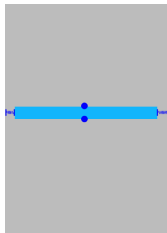
layout_width wrap_content
layout_height 48dp

Center a view horizontally



Use match_constraint

Can't use match_parent on a child view, use match_constraint instead



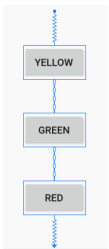
layout_width 0dp(match_constraint)
layout_height 48dp

Chains

- Let you position views in relation to each other
- Can be linked horizontally or vertically
- Provide much of LinearLayout functionality

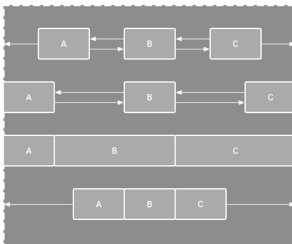
Create a Chain in Layout Editor

- Select the objects you want to be in the chain.
- Right-click and select **Chains**.
- Create a horizontal or vertical chain.



Chain styles

Adjust space between views with these different chain styles.



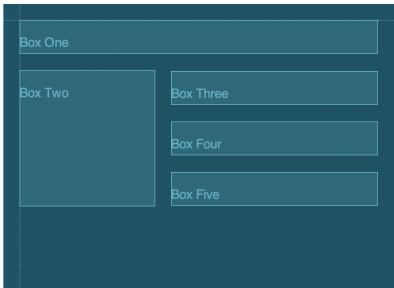
- Spread Chain
- Spread Inside Chain
- Weighted Chain
- Packed Chain

Additional topics for ConstraintLayout

Guidelines

- Let you position multiple views relative to a single guide
- Can be vertical or horizontal
- Allow for greater collaboration with design/UX teams
- Aren't drawn on the device

Guidelines in Android Studio



Example Guideline

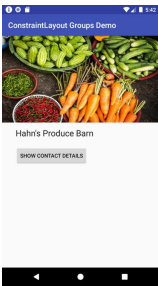
```
<ConstraintLayout>
    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/start_guideline"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_begin="16dp" />
    <TextView ...
        app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```

Creating Guidelines

- layout_constraintGuide_begin
- layout_constraintGuide_end
- layout_constraintGuide_percent

Groups

- Control the visibility of a set of widgets
- Group visibility can be toggled in code



Example group

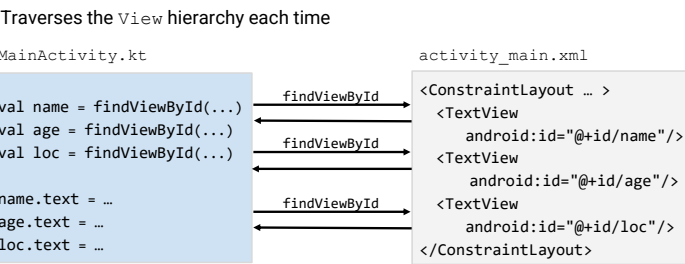
```
<androidx.constraintlayout.widget.Group
    android:id="@+id/group"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:constraint_referenced_ids="locationLabel,locationDetails"/>
```

Groups app code

```
override fun onClick(v: View?) {
    if (group.visibility == View.GONE) {
        group.visibility = View.VISIBLE
        button.setText(R.string.hide_details)
    } else {
        group.visibility = View.GONE
        button.setText(R.string.show_details)
    }
}
```

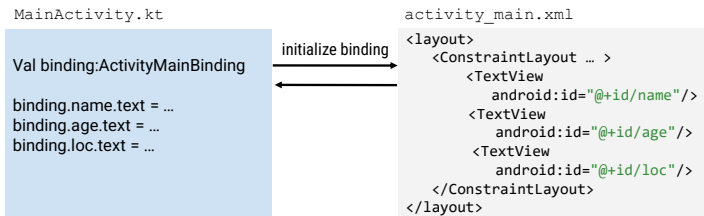
Data binding

Current approach: findViewById()



Use data binding instead

Bind UI components in your layouts to data sources in your app.



Modify build.gradle file

```
android {
  ...
  buildFeatures {
    dataBinding true
  }
}
```

Add layout tag

```
<layout>
  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView ... android:id="@+id/username" />
    <EditText ... android:id="@+id/password" />
  </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

Layout inflation with data binding

Replace this

```
setContentView(R.layout.activity_main)
```

with this

```
val binding: ActivityMainBinding = DataBindingUtil.setContentView(
    this, R.layout.activity_main)
```

```
binding.username = "Melissa"
```

Data binding layout variables

```
<layout>
  <data>
    <variable name="name" type="String"/>
  </data>
  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView
      android:id="@+id/textView"
      android:text="@{name}" />
    </androidx.constraintlayout.widget.ConstraintLayout>
  </layout>

In MainActivity.kt:
binding.name = "John"
```

Data binding layout expressions

```
<layout>
  <data>
    <variable name="name" type="String"/>
  </data>
  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView
      android:id="@+id/textView"
      android:text="@{name.toUpperCase()}" />
    </androidx.constraintlayout.widget.ConstraintLayout>
  </layout>
```

Displaying lists with RecyclerView

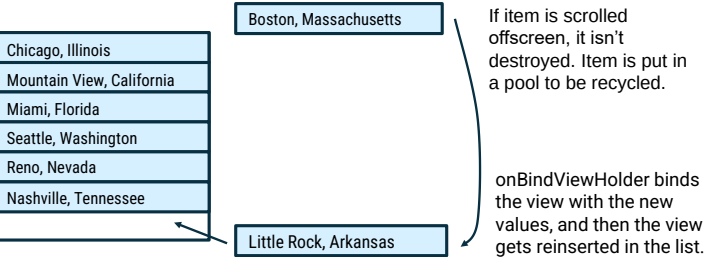
RecyclerView

- Widget for displaying lists of data
- "Recycles" (reuses) item views to make scrolling more performant
- Can specify a list item layout for each item in the dataset
- Supports animations and transitions

RecyclerView.Adapter

- Supplies data and layouts that the RecyclerView displays
- A custom Adapter extends from RecyclerView.Adapter and overrides these three functions:
 - getItemCount
 - onCreateViewHolder
 - onBindViewHolder

View recycling in RecyclerView



Add RecyclerView to your layout

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rv"
    android:scrollbars="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Create a list item layout

```
res/layout/item_view.xml
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</FrameLayout>
```

Create a list adapter

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>()
{
    class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
            parent, false)
        return MyViewHolder(layout)
    }
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = data.get(position).toString()
    }
    override fun getItemCount(): Int = data.size
}
```

Set the adapter on the RecyclerView

```
In MainActivity.kt:
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val rv: RecyclerView = findViewById(R.id.rv)
    rv.layoutManager = LinearLayoutManager(this)

    rv.adapter = MyAdapter(IntRange(0, 100).toList())
}
```

Summary

Summary

- In Lesson 5, you learned how to:
- Specify lengths in dp for your layout
 - Work with screen densities for different Android devices
 - Render Views to the screen of your app
 - Layout views within a ConstraintLayout using constraints
 - Simplify getting View references from layout with data binding
 - Display a list of text items using a RecyclerView and custom adapter

Learn more

- [Pixel density on Android](#)
- [Spacing](#)
- [Device metrics](#)
- [Type scale](#)
- [Build a Responsive UI with ConstraintLayout](#)
- [Data Binding Library](#)
- [Create dynamic lists with RecyclerView](#)

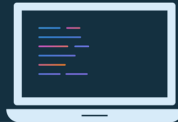
Pathway

Practice what you've learned by completing the pathway:
[Lesson 5: Layouts](#)





Lesson 6: App navigation



About this lesson

Lesson 6: App navigation

- [Multiple activities and intents](#)
- [App bar, navigation drawer, and menus](#)
- [Fragments](#)
- [Navigation in an app](#)
- [More custom navigation behavior](#)
- [Navigation UI](#)
- [Summary](#)

Multiple activities and intents

Multiple screens in an app

Sometimes app functionality may be separated into multiple screens.

Examples:

- View details of a single item (for example, product in a shopping app)
- Create a new item (for example, new email)
- Show settings for an app
- Access services in other apps (for example, photo gallery or browse documents)

Intent

Requests an action from another app component, such as another Activity

- An `Intent` usually has two primary pieces of information:
 - Action to be performed (for example, `ACTION_VIEW`, `ACTION_EDIT`, `ACTION_MAIN`)
 - Data to operate on (for example, a person's record in the contacts database)
- Commonly used to specify a request to transition to another Activity

Explicit intent

- Fulfills a request **using a specific component**
- Navigates internally to an Activity in your app
- Navigates to a specific third-party app or another app you've written

Explicit intent examples

Navigate between activities in your app:

```
fun viewNoteDetail() {  
    val intent = Intent(this, NoteDetailActivity::class.java)  
    intent.putExtra(NOTE_ID, note.id)  
    startActivity(intent)  
}
```

Navigate to a specific external app:

```
fun openExternalApp() {  
    val intent = Intent("com.example.workapp.FILE_OPEN")  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```

Implicit intent

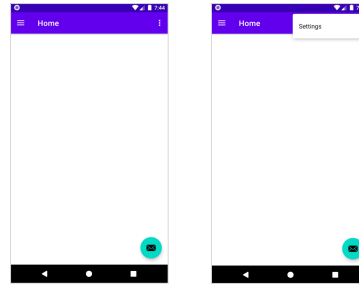
- Provides generic action the app can perform
- Resolved using mapping of the data type and action to known components
- Allows any app that matches the criteria to handle the request

Implicit intent example

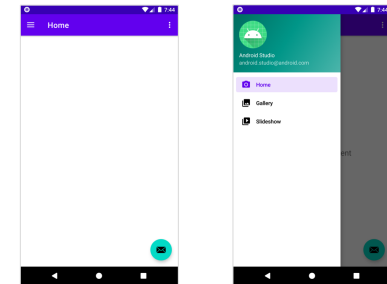
```
fun sendEmail() {  
    val intent = Intent(Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)  
    intent.putExtra(Intent.EXTRA_TEXT, "How are you?")  
  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```

App bar, navigation drawer, and menus

App bar



Navigation drawer



Menu

Define menu items in XML menu resource (located in `res/menu` folder)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
  <item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:title="@string/action_settings"
    app:showAsAction="never" />
</menu>
```

More menu options

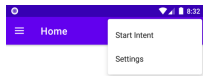
```
<menu>
  <group android:checkableBehavior="single">
    <item
      android:id="@+id/nav_home"
      android:icon="@drawable/ic_menu_camera"
      android:title="@string/menu_home" />
    <item
      android:id="@+id/nav_gallery"
      android:icon="@drawable/ic_menu_gallery"
      android:title="@string/menu_gallery" />
    <item
      android:id="@+id/nav_slideshow"
      android:icon="@drawable/ic_menu_slideshow"
      android:title="@string/menu_slideshow" />
  </group>
</menu>
```

Options menu example

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

  <item android:id="@+id/action_intent"
    android:title="@string/action_intent" />

  <item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:title="@string/action_settings"
    app:showAsAction="never" />
</menu>
```



Inflate options menu

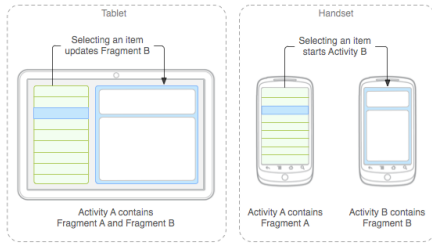
```
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    menuInflater.inflate(R.menu.main, menu)
    return true
}
```

Handle menu options selected

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when (item.itemId) {
        R.id.action_intent -> {
            val intent = Intent(Intent.ACTION_WEB_SEARCH)
            intent.putExtra(SearchManager.QUERY, "pizza")
            if (intent.resolveActivity(packageManager) != null) {
                startActivity(intent)
            }
        }
        else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()
    }
    ...
}
```

Fragments

Fragments for tablet layouts



Fragment

- Represents a behavior or portion of the UI in an activity ("microactivity")
- Must be hosted in an activity
- Lifecycle tied to host activity's lifecycle
- Can be added or removed at runtime

Note about fragments

Use the **AndroidX** version of the `Fragment` class (`androidx.fragment.app.Fragment`).

Don't use the **platform** version of the `Fragment` class (`android.app.Fragment`), which was deprecated.

Navigation within an app

Navigation component

- Collection of libraries and tooling, including an integrated editor, for creating navigation paths through an app
- Assumes one `Activity` per graph with many `Fragment` destinations
- Consists of three major parts:
 - Navigation graph
 - Navigation Host (`NavHost`)
 - Navigation Controller (`NavController`)

Add dependencies

In `build.gradle`, under dependencies:

```
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"
```

```
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

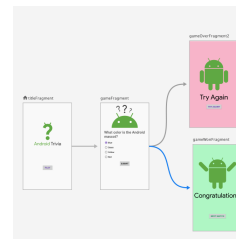
Navigation host (NavHost)

```
<fragment
    android:id="@+id/nav_host"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph_name"/>
```

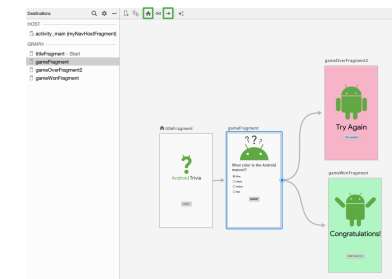
Navigation graph

New resource type located in `res/navigation` directory

- XML file containing all of your navigation destinations and actions
- Lists all the (Fragment/Activity) destinations that can be navigated to
- Lists the associated actions to traverse between them
- Optionally lists animations for entering or exiting



Navigation Editor in Android Studio



Creating a Fragment

- Extend `Fragment` class
- Override `onCreateView()`
- Inflate a layout for the Fragment that you have defined in XML

```
class DetailFragment : Fragment() {  
  
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?): View? {  
        return inflater.inflate(R.layout.detail_fragment, container, false)  
    }  
}
```

Specifying Fragment destinations

- Fragment destinations are denoted by the `action` tag in the navigation graph.
- Actions can be defined in XML directly or in the Navigation Editor by dragging from source to destination.
- Autogenerated action IDs take the form of `action_<sourceFragment>_to_<destinationFragment>`.

Example fragment destination

```
<fragment  
    android:id="@+id/welcomeFragment"  
    android:name="com.example.android.navigation.WelcomeFragment"  
    android:label="fragment_welcome"  
    tools:layout="@layout/fragment_welcome" >  
  
    <action  
        android:id="@+id/action_welcomeFragment_to_detailFragment"  
        app:destination="@id/detailFragment" />  
  
</fragment>
```

Navigation Controller (NavController)

`NavController` manages UI navigation in a navigation host.

- Specifying a destination path only names the action, but it doesn't execute it.
- To follow a path, use `NavController`.

Example NavController

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val navController = findNavController(R.id.myNavHostFragment)  
    }  
  
    fun navigateToDetail() {  
        navController.navigate(R.id.action_welcomeFragment_to_detailFragment)  
    }  
}
```

More custom navigation behavior

Passing data between destinations

Using Safe Args:

- Ensures arguments have a valid type
- Lets you provide default values
- Generates a `<SourceDestination>Directions` class with methods for every action in that destination
- Generates a class to set arguments for every named action
- Generates a `<TargetDestination>Args` class providing access to the destination's arguments

Setting up Safe Args

In the project `build.gradle` file:

```
buildscript {  
    repositories {  
        google()  
    }  
    dependencies {  
        classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"  
    }  
}
```

In the app's or module's `build.gradle` file:

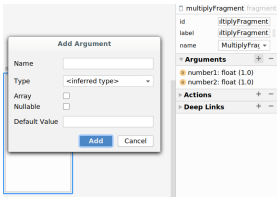
```
apply plugin: "androidx.navigation.safeargs.kotlin"
```

Sending data to a Fragment

1. Create arguments the destination fragment will expect.
2. Create action to link from source to destination.
3. Set the arguments in the action method on `<Source>FragmentDirections`.
4. Navigate according to that action using the Navigation Controller.
5. Retrieve the arguments in the destination fragment.

Destination arguments

```
<fragment
  android:id="@+id/multiplyFragment"
  android:name="com.example.arithmetic.MultiplyFragment"
  android:label="MultiplyFragment" >
  <argument
    android:name="number1"
    app:argType="float"
    android:defaultValue="1.0" />
  <argument
    android:name="number2"
    app:argType="float"
    android:defaultValue="1.0" />
</fragment>
```



Supported argument types

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Integer	"integer"	Yes	No
Float	"float"	Yes	No
Long	"long"	Yes	No
Boolean	"boolean"	Yes ("true" or "false")	No
String	"string"	Yes	Yes
Array	above type + "[]" (for example, "string[]" "long[]")	Yes (only "@null")	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	"reference"	Yes	No

Supported argument types: Custom classes

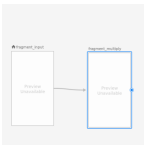
Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Serializable	Fully qualified class name	Yes (only "@null")	Yes
Parcelable	Fully qualified class name	Yes (only "@null")	Yes

Create action from source to destination

```
In nav_graph.xml:
<fragment
  android:id="@+id/fragment_input"
  android:name="com.example.arithmetic.InputFragment">

  <action
    android:id="@+id/action_to_multiplyFragment"
    app:destination="@id/multiplyFragment" />

</fragment>
```



Navigating with actions

```
In InputFragment.kt:
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    binding.button.setOnClickListener {
        val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0
        val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0

        val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)
        view.findNavController().navigate(action)
    }
}
```

Retrieving Fragment arguments

```
class MultiplyFragment : Fragment() {
    val args: MultiplyFragmentArgs by navArgs()
    lateinit var binding: FragmentMultiplyBinding
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val number1 = args.number1
        val number2 = args.number2
        val result = number1 * number2
        binding.output.text = "${number1} * ${number2} = ${result}"
    }
}
```

Navigation UI

Menus revisited

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    val navController = findNavController(R.id.nav_host_fragment)
    return item.onNavDestinationSelected(navController) ||
        super.onOptionsItemSelected(item)
}
```

DrawerLayout for navigation drawer

```
<androidx.drawerlayout.widget.DrawerLayout
  android:id="@+id/drawer_layout" ...>

  <fragment
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:id="@+id/nav_host_fragment" ... />

  <com.google.android.material.navigation.NavigationView
    android:id="@+id/nav_view"
    app:menu="@menu/activity_main_drawer" ... />

</androidx.drawerlayout.widget.DrawerLayout>
```

Finish setting up navigation drawer

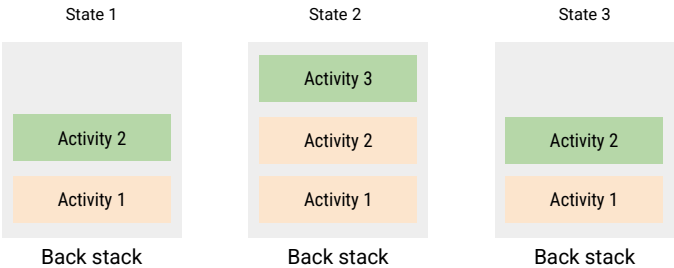
Connect `DrawerLayout` to navigation graph:

```
val appBarConfiguration = AppBarConfig(navController.graph, drawer)
```

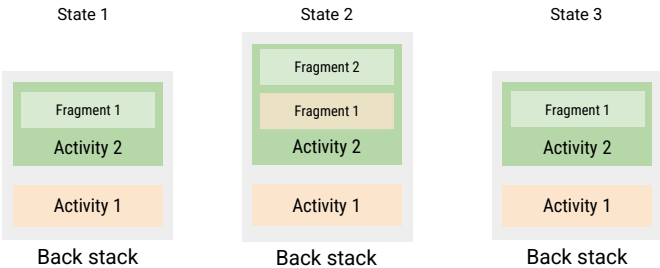
Set up `NavigationView` for use with a `NavController`:

```
val navView = findViewById<NavigationView>(R.id.nav_view)
navView.setupWithNavController(navController)
```

Understanding the back stack



Fragments and the back stack



Summary

Summary

- In Lesson 6, you learned how to:
- Use explicit and implicit intents to navigate between Activities
 - Structure apps using fragments instead of putting all UI code in the Activity
 - Handle navigation with `NavGraph`, `NavHost`, and `NavController`
 - Use `Safe Args` to pass data between fragment destinations
 - Use `NavigationUI` to hook up top app bar, navigation drawer, and bottom navigation
 - Android keeps a back stack of all the destinations you've visited, with each new destination being pushed onto the stack.

Learn more

- [Principles of navigation](#)
- [Navigation component](#)
- [Pass data between destinations](#)
- [NavigationUI](#)

Pathway

Practice what you've learned by completing the pathway:

[Lesson 6: App navigation](#)





Lesson 7: Activity and fragment lifecycles

About this lesson

Lesson 7: Activity and fragment lifecycles

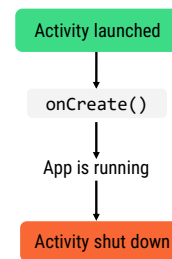
- [Activity lifecycle](#)
- [Logging](#)
- [Fragment lifecycle](#)
- [Lifecycle-aware components](#)
- [Tasks and back stack](#)
- [Summary](#)

Activity lifecycle

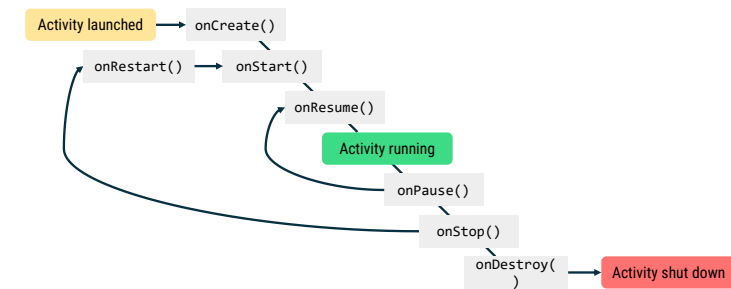
Why it matters

- Preserve user data and state if:
 - User temporarily leaves app and then returns
 - User is interrupted (for example, a phone call)
 - User rotates device
- Avoid memory leaks and app crashes.

Simplified activity lifecycle



Activity lifecycle



Activity states



onCreate()

- Activity is created and other initialization work occurs
- You must implement this callback
- Inflate activity UI and perform other app startup logic

onStart()

- Activity becomes visible to the user
- Called after activity:
 - `onCreate()`
 - or
 - `onRestart()` if activity was previously stopped

onResume()

- Activity gains input focus:
 - User can interact with the activity
- Activity stays in resumed state until system triggers activity to be paused

onPause()

- Activity has lost focus (not in foreground)
- Activity is still visible, but user is not actively interacting with it
- Counterpart to `onResume()`

onStop()

- Activity is no longer visible to the user
- Release resources that aren't needed anymore
- Save any persistent state that the user is in the process of editing so they don't lose their work

onDestroy()

- Activity is about to be destroyed, which can be caused by:
 - Activity has finished or been dismissed by the user
 - Configuration change
- Perform any final cleanup of resources.
- Don't rely on this method to save user data (do that earlier)

Summary of activity states

State	Callbacks	Description
Created	<code>onCreate()</code>	Activity is being initialized.
Started	<code>onStart()</code>	Activity is visible to the user.
Resumed	<code>onResume()</code>	Activity has input focus.
Paused	<code>onPause()</code>	Activity does not have input focus.
Stopped	<code>onStop()</code>	Activity is no longer visible.
Destroyed	<code>onDestroy()</code>	Activity is destroyed.

Save state

User expects UI state to stay the same after a config change or if the app is terminated when in the background.

- Activity is destroyed and restarted, or app is terminated and activity is started.
- Store user data needed to reconstruct app and activity Lifecycle changes:
 - Use `Bundle` provided by `onSaveInstanceState()`.
 - `onCreate()` receives the `Bundle` as an argument when activity is created again.

Logging

Logging in Android

- Monitor the flow of events or state of your app.
- Use the built-in `Log` class or third-party library.
- Example `Log` method call: `Log.d(TAG, "Message")`

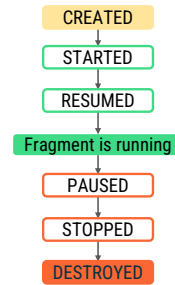
```
Log.d
Emulator Pixel_2_Oreo_API_26 - com.example.myapplication:50 - Verbose - MainActivity
Log.d: 2020-05-26 15:19:42.111 5825-5825/com.example.myapplication I/zygote: at void com.example.myapplication.MainActivity.onCreate(android.os.Bundle) (MainActivity.kt:14)
2020-05-26 15:19:42.249 5825-5825/com.example.myapplication I/mainactivity: onStart
2020-05-26 15:19:42.295 5825-5825/com.example.myapplication I/mainactivity: onResume
2020-05-26 15:19:52.837 5825-5825/com.example.myapplication I/mainactivity: onPause
```

Write logs

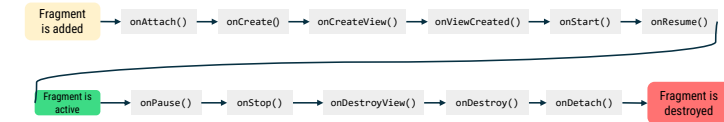
Priority level	Log method
Verbose	<code>Log.v(String, String)</code>
Debug	<code>Log.d(String, String)</code>
Info	<code>Log.i(String, String)</code>
Warning	<code>Log.w(String, String)</code>
Error	<code>Log.e(String, String)</code>

Fragment lifecycle

Fragment states



Fragment lifecycle diagram



onAttach()

- Called when a fragment is attached to a context
- Immediately precedes `onCreate()`

onCreateView()

- Called to create the view hierarchy associated with the fragment
- Inflate the fragment layout here and return the root view

onViewCreated()

- Called when view hierarchy has already been created
- Perform any remaining initialization here (for example, restore state from `Bundle`)

onDestroyView() and onDetach()

- `onDestroyView()` is called when view hierarchy of fragment is removed.
- `onDetach()` is called when fragment is no longer attached to the host.

Summary of fragment states

State	Callbacks	Description
Initialized	<code>onAttach()</code>	Fragment is attached to host.
Created	<code>onCreate()</code> , <code>onCreateView()</code> , <code>onViewCreated()</code>	Fragment is created and layout is being initialized.
Started	<code>onStart()</code>	Fragment is started and visible.
Resumed	<code>onResume()</code>	Fragment has input focus.
Paused	<code>onPause()</code>	Fragment no longer has input focus.
Stopped	<code>onStop()</code>	Fragment is not visible.
Destroyed	<code>onDestroyView()</code> , <code>onDestroy()</code> , <code>onDetach()</code>	Fragment is removed from host.

Save fragment state across config changes

Preserve UI state in fragments by storing state in `Bundle`:

- `onSaveInstanceState(outState: Bundle)`

Retrieve that data by receiving the `Bundle` in these fragment callbacks:

- `onCreate()`
- `onCreateView()`
- `onViewCreated()`

Lifecycle-aware components

Lifecycle-aware components

Adjust their behavior based on activity or fragment lifecycle

- Use the `androidx.lifecycle` library
- `Lifecycle` tracks the lifecycle state of an activity or fragment
 - Holds current lifecycle state
 - Dispatches lifecycle events (when there are state changes)

LifecycleOwner

- Interface that says this class has a lifecycle
- Implementers must implement `getLifecycle()` method

Examples: `Fragment` and `AppCompatActivity` are implementations of `LifecycleOwner`

LifecycleObserver

Implement `LifecycleObserver` interface:

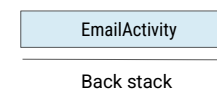
```
class MyObserver : LifecycleObserver {  
    @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)  
    fun connectListener() {  
        ...  
    }  
}
```

Add the observer to the lifecycle:

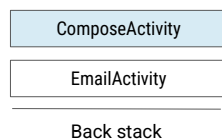
```
myLifecycleOwner.getLifecycle().addObserver(MyObserver())
```

Tasks and back stack

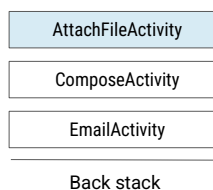
Back stack of activities



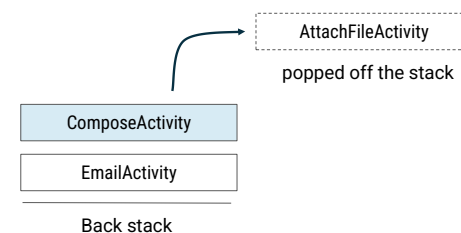
Add to the back stack



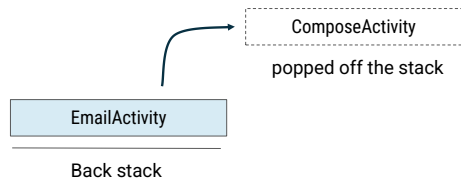
Add to the back stack again



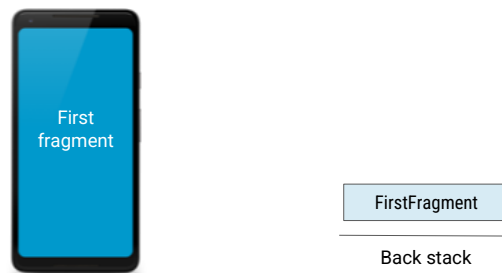
Tap Back button



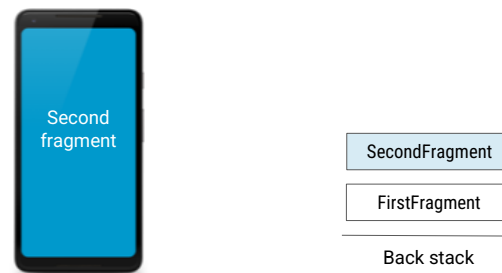
Tap Back button again



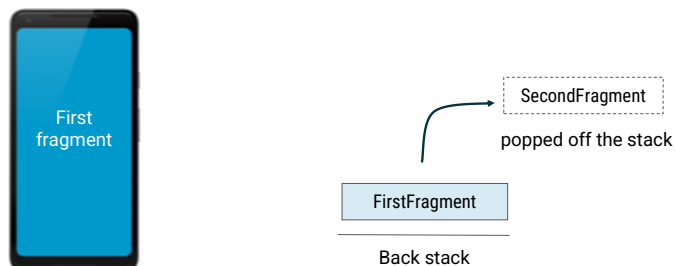
First destination in the back stack



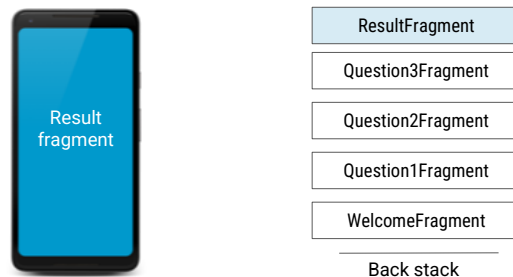
Add a destination to the back stack



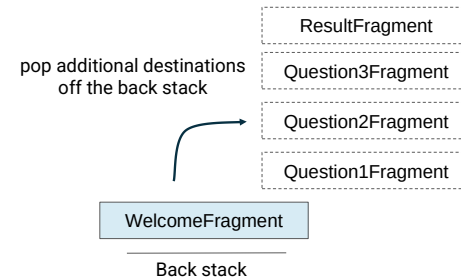
Tap Back button



Another back stack example



Modify Back button behavior



Summary

Summary

In Lesson 7, you learned how to:

- Understand how an activity instance transitions through different lifecycle states as the user interacts with or leaves your app
- Reserve UI state across configuration changes using a `Bundle`
- Fragment lifecycle callback methods similar to activity, but with additions
- Use lifecycle-aware components help organize your app code
- Use default or custom back stack behavior
- Use logging to help debug and track the state of the app

Learn more

- [Understand the Activity Lifecycle](#)
- [Activity class](#)
- [Fragments guide and lifecycle](#)
- [Fragment class](#)

Pathway

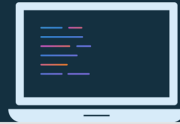
Practice what you've learned by completing the pathway:

[Lesson 7: Activity and Fragment Lifecycles](#)





Lesson 8: App architecture (UI layer)



About this lesson

Lesson 8: App architecture (UI layer)

- [Android app architecture](#)
- [ViewModel](#)
- [Data binding](#)
- [LiveData](#)
- [Transform LiveData](#)
- [Summary](#)

Android app architecture

Avoid short-term hacks

- External factors, such as tight deadlines, can lead to poor decisions about app design and structure.
- Decisions have consequences for future work (app can be harder to maintain long-term).
- Need to balance on-time delivery and future maintenance burden.

Examples of short-term hacks

- Tailoring your app to a specific device
- Blindly copying and pasting code into your files
- Placing all business logic in activity file
- Hardcoding user-facing strings in your code

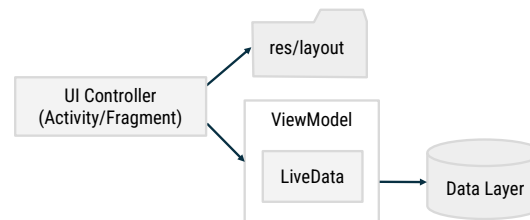
Why you need good app architecture

- Clearly defines where specific business logic belongs
- Makes it easier for developers to collaborate
- Makes your code easier to test
- Lets you benefit from already-solved problems
- Saves time and reduces technical debt as you extend your app

Android Jetpack

- Android libraries that incorporate best practices and provide backward compatibility in your apps
- Jetpack comprises the `androidx.*` package libraries

Separation of concerns



Architecture components

- Architecture design patterns, like MVVM and MVI, describe a loose template for what the structure of your app should be.
- Jetpack architecture components help you design robust, testable, and maintainable apps.

ViewModel

Gradle: lifecycle extensions

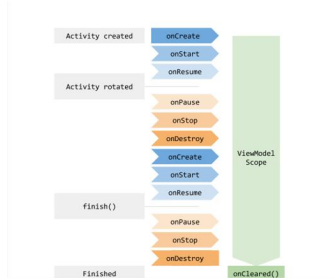
In app/build.gradle file:

```
dependencies {  
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"  
    implementation "androidx.activity:activity-ktx:$activity_version"  
}
```

ViewModel

- Prepares data for the UI
- Must not reference activity, fragment, or views in view hierarchy
- Scoped to a lifecycle (which activity and fragment have)
- Enables data to survive configuration changes
- Survives as long as the scope is alive

Lifetime of a ViewModel



Kabaddi Kounter



ViewModel class

```
abstract class ViewModel
```

Summary

Public constructors	
<code><init>()</code>	
ViewModel is a class that is responsible for preparing and managing the data for an Activity or a Fragment .	
Protected methods	
open Unit	<code>onCleared()</code>
This method will be called when this ViewModel is no longer used and will be destroyed.	
Extension properties	
From androidx.lifecycle	
CoroutineScope	viewModelScope
CoroutineScope tied to this ViewModel .	

Implement a ViewModel

```
class ScoreViewModel : ViewModel() {  
    var scoreA : Int = 0  
    var scoreB : Int = 0  
    fun incrementScore(isTeamA: Boolean) {  
        if (isTeamA) {  
            scoreA++  
        }  
        else {  
            scoreB++  
        }  
    }  
}
```

Load and use a ViewModel

```
class MainActivity : AppCompatActivity() {  
    // Delegate provided by androidx.activity.viewModels  
    val viewModel: ScoreViewModel by viewModels()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val scoreViewA: TextView = findViewById(R.id.scoreA)  
        scoreViewA.text = viewModel.scoreA.toString()  
    }  
}
```

Using a ViewModel

Within MainActivity onCreate():

```
val scoreViewA: TextView = findViewById(R.id.scoreA)  
val plusOneButtonA: Button = findViewById(R.id.plusOne_teamA)  
  
plusOneButtonA.setOnClickListener {  
    viewModel.incrementScore(true)  
    scoreViewA.text = viewModel.scoreA.toString()  
}
```

Data binding

ViewModels and data binding

- App architecture without data binding



- ViewModels can work in concert with data binding



Data binding in XML revisited

Specify ViewModels in the `data` tag of a binding.

```
<layout>
  <data>
    <variable>
      name="viewModel"
      type="com.example.kabaddikounter.ScoreViewModel" />
    </data>
  <ConstraintLayout .../>
</layout>
```

Attaching a ViewModel to a data binding

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
            R.layout.activity_main)

        binding.viewModel = viewModel
        ...
    }
}
```

Using a ViewModel from a data binding

In `activity_main.xml`:

```
<TextView
    android:id="@+id/scoreViewA"
    android:text="@{viewModel.scoreA.toString()}" />
...

```

ViewModels and data binding

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
        R.layout.activity_main)

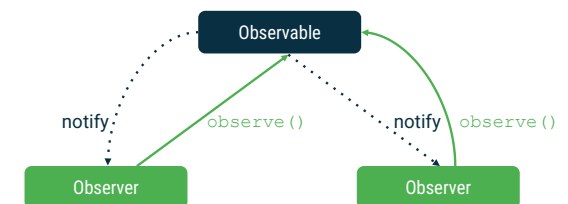
    binding.plusOneButtonA.setOnClickListener {
        viewModel.incrementScore(true)
        binding.scoreViewA.text = viewModel.scoreA.toString()
    }
}
```

LiveData

Observer design pattern

- Subject maintains list of observers to notify when state changes.
- Observers receive state changes from subject and execute appropriate code.
- Observers can be added or removed at any time.

Observer design pattern diagram



- A lifecycle-aware data holder that can be observed
- Wrapper that can be used with any data including lists (for example, `LiveData<Int>` holds an `Int`)
- Often used by ViewModels to hold individual data fields
- Observers (activity or fragment) can be added or removed
 - `observe(owner: LifecycleOwner, observer: Observer)`
 - `removeObserver(observer: Observer)`

LiveData<T>	MutableLiveData<T>
• <code>getValue()</code>	• <code>getValue()</code> • <code>postValue(value: T)</code> • <code>setValue(value: T)</code>

T is the type of data that's stored in `LiveData` or `MutableLiveData`.

```
class ScoreViewModel : ViewModel() {

    private val _scoreA = MutableLiveData<Int>()
    val scoreA: LiveData<Int>
        get() = _scoreA

    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        }
        ...
    }
}
```

Add an observer on LiveData

Set up click listener to increment `ViewModel` score:

```
binding.plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
}
```

Create observer to update team A score on screen:

```
val scoreA_Observer = Observer<Int> { newValue ->
    binding.scoreViewA.text = newValue.toString()
}
```

Add the observer onto `scoreA` `LiveData` in `ViewModel`:

```
viewModel.scoreA.observe(this, scoreA_Observer)
```

Two-way data binding

- We already have two-way binding with `ViewModel` and `LiveData`.
- Binding to `LiveData` in XML eliminates need for an observer in code.

Example layout XML

```
<layout>
    <data>
        <variable>
            name="viewModel"
            type="com.example.kabaddikounter.ScoreViewModel" />
        </data>

    <ConstraintLayout ..>
        <TextView ...
            android:id="@+id/scoreViewA"
            android:text="@{viewModel.scoreA.toString()}" />
        ...
    </ConstraintLayout>
</layout>
```

Example Activity

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil
            .setContentView(this, R.layout.activity_main)

        binding.viewModel = viewModel
        binding.lifecycleOwner = this

        binding.plusOneButtonA.setOnClickListener {
            viewModel.incrementScore(true)
        }
        ...
    }
}
```

Example ViewModel

```
class ScoreViewModel : ViewModel() {
    private val _scoreA = MutableLiveData<Int>()
    val scoreA: LiveData<Int>
        get() = _scoreA
    private val _scoreB = MutableLiveData<Int>()
    val scoreB: LiveData<Int>
        get() = _scoreB
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        } else {
            _scoreB.value = _scoreB.value!! + 1
        }
    }
}
```

Transform LiveData

Summary

LiveData can be transformed into a new LiveData object.

- map()
- switchMap()

```
val result: LiveData<String> = Transformations.map(viewModel.scoreA) {
    x -> if (x > 10) "A Wins" else ""
}
```

Summary

In Lesson 8, you learned how to:

- Follow good app architecture design, and the separation-of-concerns principle to make apps more maintainable and reduce technical debt
- Create a ViewModel to hold data separately from a UI controller
- Use ViewModel with data binding to make a responsive UI with less code
- Use observers to automatically get updates from LiveData

Learn More

- [Guide to app architecture](#)
- [Android Jetpack](#)
- [ViewModel Overview](#)
- [Android architecture sample app](#)
- [ViewModelProvider](#)
- [Lifecycle Aware Data Loading with Architecture Components](#)
- [ViewModels and LiveData: Patterns + AntiPatterns](#)

Pathway

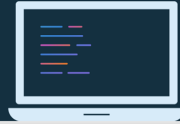
Practice what you’ve learned by completing the pathway:

[Lesson 8: App architecture \(UI layer\)](#)





Lesson 9: App architecture (persistence)



About this lesson

Lesson 9: App architecture (persistence)

- [Storing data](#)
- [Room persistence library](#)
- [Asynchronous programming](#)
- [Coroutines](#)
- [Testing databases](#)
- [Summary](#)

Storing data

Ways to store data in an Android app

- App-specific storage
- Shared storage (files to be shared with other apps)
- Preferences
- Databases

What is a database?

Collection of structured data that can be easily accessed, searched, and organized, consisting of:

- Tables
- Rows
- Columns

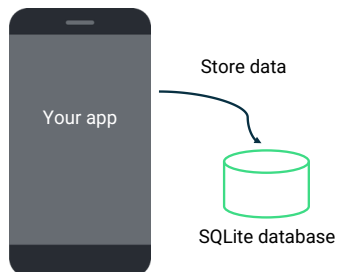


Structured Query Language (SQL)

Use SQL to access and modify a relational database.

- Create new tables
- Query for data
- Insert new data
- Update data
- Delete data

SQLite in Android



Example SQLite commands

Create `INSERT INTO colors VALUES ("red", "#FF0000");`

Read `SELECT * from colors;`

Update `UPDATE colors SET hex="#DD0000" WHERE name="red";`

Delete `DELETE FROM colors WHERE name = "red";`

Interacting directly with a database

- No compile-time verification of raw SQL queries
- Need lots of boilerplate code to convert between SQL queries ↔ data objects

Room persistence library

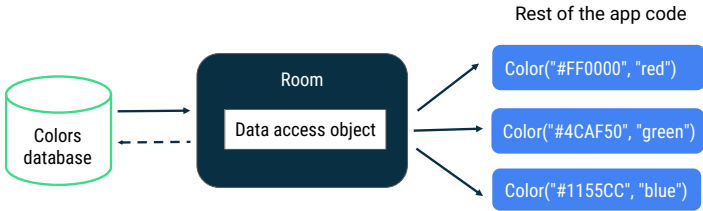
Add Gradle dependencies

```
dependencies {
    implementation "androidx.room:room-runtime:$room_version"
    kapt "androidx.room:room-compiler:$room_version"

    // Kotlin Extensions and Coroutines support for Room
    implementation "androidx.room:room-ktx:$room_version"

    // Test helpers
    testImplementation "androidx.room:room-testing:$room_version"
}
```

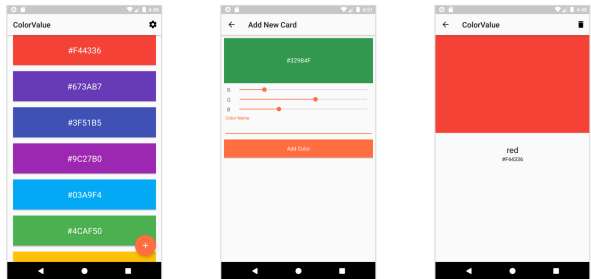
Room



ColorValue app

Room

Color class



- Entity
- DAO
- Database

Color
ColorDao
ColorDatabase

```
data class Color {
    val hex: String,
    val name: String
}
```

Annotations

Entity

Example entity

- Provide extra information to the compiler
@Entity marks entity class, @Dao for DAO, @Database for database
- Can take parameters
@Entity(tableName = "colors")
- Can autogenerate code for you

Class that maps to a SQLite database table

- @Entity
- @PrimaryKey
- @ColumnInfo

```
@Entity(tableName = "colors")
data class Color {
    @PrimaryKey(autoGenerate = true) val _id: Int,
    @ColumnInfo(name = "hex_color") val hex: String,
    val name: String
}
```

colors
_id
hex_color
name

Data access object (DAO)

Work with DAO classes instead of accessing database directly:

- Define database interactions in the DAO.
- Declare DAO as an interface or abstract class.
- Room creates DAO implementation at compile time.
- Room verifies all of your DAO queries at compile-time.

Example DAO

```
@Dao
interface ColorDao {
    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>

    @Insert
    fun insert(vararg color: Color)

    @Update
    fun update(color: Color)

    @Delete
    fun delete(color: Color)
```

Query

```
@Dao
interface ColorDao {

    @Query("SELECT * FROM colors")
    fun getAll(): Array<Color>

    @Query("SELECT * FROM colors WHERE name = :name")
    fun getColorByName(name: String): LiveData<Color>

    @Query("SELECT * FROM colors WHERE hex_color = :hex")
    fun getColorByHex(hex: String): LiveData<Color>
```

Insert

```
@Dao
interface ColorDao {
    ...

    @Insert
    fun insert(vararg color: Color)

    ...
}
```

Update

```
@Dao
interface ColorDao {
    ...

    @Update
    fun update(color: Color)

    ...
}
```

Delete

```
@Dao
interface ColorDao {
    ...

    @Delete
    fun delete(color: Color)

    ...
}
```

Create a Room database

- Annotate class with `@Database` and include list of entities:
`@Database(entities = [Color::class], version = 1)`
- Declare abstract class that extends `RoomDatabase`:
`abstract class ColorDatabase : RoomDatabase() {`
 - Declare abstract method with no args that returns the DAO:
`abstract fun colorDao(): ColorDao`

Example Room database

```
@Database(entities = [Color::class], version = 1)
abstract class ColorDatabase : RoomDatabase() {
    abstract fun colorDao(): ColorDao

    companion object {
        @Volatile
        private var INSTANCE: ColorDatabase? = null

        fun getInstance(context: Context): ColorDatabase {
            ...
        }
    }
    ...
}
```

Create database instance

```
fun getInstance(context: Context): ColorDatabase {
    return INSTANCE ?: synchronized(this) {
        INSTANCE ?: Room.databaseBuilder(
            context.applicationContext,
            ColorDatabase::class.java, "color_database"
        )
            .fallbackToDestructiveMigration()
            .build()
            .also { INSTANCE = it }
    }
}
```

Get and use a DAO

Get the DAO from the database:

```
val colorDao = ColorDatabase.getInstance(application).colorDao()
```

Create new Color and use DAO to insert it into database:

```
val newColor = Color(hex = "#6200EE", name = "purple")
colorDao.insert(newColor)
```

Asynchronous programming

Long-running tasks

- Download information
- Sync with a server
- Write to a file
- Heavy computation
- Read from, or write to, a database

Need for async programming

- Limited time to do tasks and remain responsive
- Balanced with the need to execute long-running tasks
- Control over how and where tasks are executed

Async programming on Android

- Threading
- Callbacks
- Plus many other options

What is the recommended way?

Coroutines

Coroutines

Benefits of coroutines

- Keep your app responsive while managing long-running tasks.
- Simplify asynchronous code in your Android app.
- Write code in sequential way
- Handle exceptions with `try/catch` block

- Lightweight
- Fewer memory leaks
- Built-in cancellation support
- Jetpack integration

Suspend functions

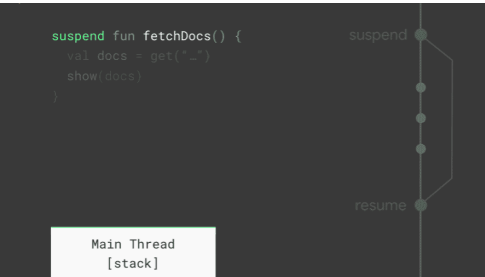
- Add `suspend` modifier
- Must be called by other `suspend` functions or coroutines

```
suspend fun insert(word: Word) {
    wordDao.insert(word)
}
```

Suspend and resume

- suspend
Pauses execution of current coroutine and saves local variables
- resume
Automatically loads saved state and continues execution from the point the code was suspended

Example



Add suspend modifier to DAO methods

```
@Dao
interface ColorDao {
    @Query("SELECT * FROM colors")
    suspend fun getAll(): Array<Color>

    @Insert
    suspend fun insert(vararg color: Color)

    @Update
    suspend fun update(color: Color)

    @Delete
    suspend fun delete(color: Color)
```

Control where coroutines run

Dispatcher	Description of work	Examples of work
Dispatchers.Main	UI and nonblocking (short) tasks	Updating LiveData, calling suspend functions
Dispatchers.IO	Network and disk tasks	Database, file IO
Dispatchers.Default	CPU intensive	Parsing JSON

withContext

```
suspend fun get(url: String) {
    // Start on Dispatchers.Main

    withContext(Dispatchers.IO) {
        // Switches to Dispatchers.IO
        // Perform blocking network IO here
    }

    // Returns to Dispatchers.Main
}
```

CoroutineScope

- Coroutines must run in a CoroutineScope:
- Keeps track of all coroutines started in it (even suspended ones)
 - Provides a way to cancel coroutines in a scope
 - Provides a bridge between regular functions and coroutines
- Examples:
- GlobalScope
 - ViewModel has viewModelScope
 - Lifecycle has lifecycleScope

Start new coroutines

- launch - no result needed
`fun loadUI() {
 launch {
 fetchDocs()
 }
}`
- async - can return a result

ViewModelScope

```
class MyViewModel: ViewModel() {
    init {
        viewModelScope.launch {
            // Coroutine that will be canceled
            // when the ViewModel is cleared
        }
    }
    ...
}
```

Example viewModelScope

```
class ColorViewModel(val dao: ColorDao, application: Application)
: AndroidViewModel(application) {
    fun save(color: Color) {
        viewModelScope.launch {
            colorDao.insert(color)
        }
    }
    ...
}
```

Testing databases

Add Gradle dependencies

```
android {
    defaultConfig {
        ...
        testInstrumentationRunner "androidx.test.runner
        .AndroidJUnitRunner"
        testInstrumentationRunnerArguments clearPackageData: 'true'
    }
}

dependencies {
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test.ext:junit:1.1.0'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.1.1'
}
```

Testing Android code

- `@RunWith(AndroidJUnit4::class)`
- `@Before`
- `@After`
- `@Test`

Create test class

```
@RunWith(AndroidJUnit4::class)
class DatabaseTest {

    private lateinit val colorDao: ColorDao
    private lateinit val db: ColorDatabase

    private val red = Color(hex = "#FF0000", name = "red")
    private val green = Color(hex = "#00FF00", name = "green")
    private val blue = Color(hex = "#0000FF", name = "blue")

    ...
}
```

Create and close database for each test

```
In DatabaseTest.kt:

@Before
fun createDb() {
    val context: Context = ApplicationProvider.getApplicationContext()
    db = Room.inMemoryDatabaseBuilder(context, ColorDatabase::class.java)
        .allowMainThreadQueries()
        .build()
    colorDao = db.colorDao()
}

@After
@Throws(IOException::class)
fun closeDb() = db.close()
```

Test insert and retrieve from a database

```
In DatabaseTest.kt:

@Test
@Throws(Exception::class)
fun insertAndRetrieve() {
    colorDao.insert(red, green, blue)
    val colors = colorDao.getAll()
    assert(colors.size == 3)
}
```

Summary

Summary

In Lesson 9, you learned how to:

- Set up and configure a database using the Room library
- Use coroutines for asynchronous programming
- Use coroutines with Room
- Test a database

Learn more

- [7 Pro-tips for Room](#)
- [Room Persistence Library](#)
- [SQLite Home Page](#)
- [Save data using SQLite](#)
- [Coroutines Guide](#)
- [Dispatchers - kotlinx-coroutines-core](#)
- [Coroutines on Android \(part I\): Getting the background](#)
- [Coroutines on Android \(part II\): Getting started](#)
- [Easy Coroutines in Android: viewModelScope](#)
- [Kotlin Coroutines 101](#)

Pathway

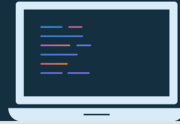
Practice what you've learned by completing the pathway:

[Lesson 9: App architecture \(persistence\)](#)





Lesson 10: Advanced RecyclerView use cases



About this lesson

Lesson 10: Advanced RecyclerView use cases

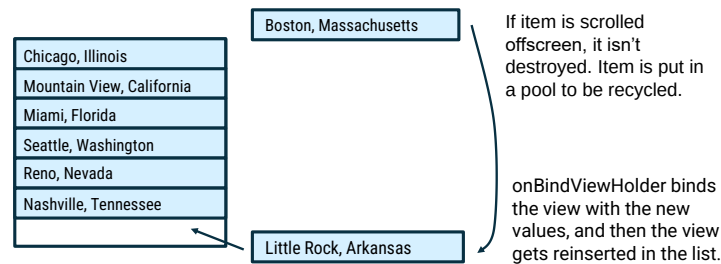
- [RecyclerView recap](#)
- [Advanced binding](#)
- [Multiple item view types](#)
- [Headers](#)
- [Grid layout](#)
- [Summary](#)

RecyclerView recap

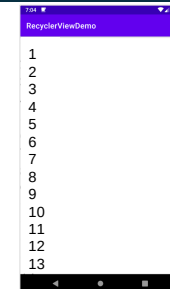
RecyclerView overview

- Widget for displaying lists of data
- "Recycles" (reuses) item views to make scrolling more performant
- Can specify a list item layout for each item in the dataset
- Supports animations and transitions

View recycling in RecyclerView



RecyclerViewDemo app



Adapter for RecyclerViewDemo

```
class NumberListAdapter(var data: List<Int>):  
    RecyclerView.Adapter<NumberListAdapter.IntViewHolder>() {  
    class IntViewHolder(val row: View): RecyclerView.ViewHolder(row) {  
        val textView = row.findViewById<TextView>(R.id.number)  
    }  
}
```

Functions for RecyclerViewDemo

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
    IntViewHolder {  
    val layout = LayoutInflater.from(parent.context)  
        .inflate(R.layout.item_view, parent, false)  
    return IntViewHolder(layout)  
}  
  
override fun onBindViewHolder(holder: IntViewHolder, position: Int) {  
    holder.textView.text = data.get(position).toString()  
}
```

Set the adapter onto the RecyclerView

In MainActivity.kt:

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val rv: RecyclerView = findViewById(R.id.rv)  
    rv.layoutManager = LinearLayoutManager(this)  
  
    rv.adapter = NumberListAdapter(IntRange(0,100).toList())  
}
```

Make items in the list clickable

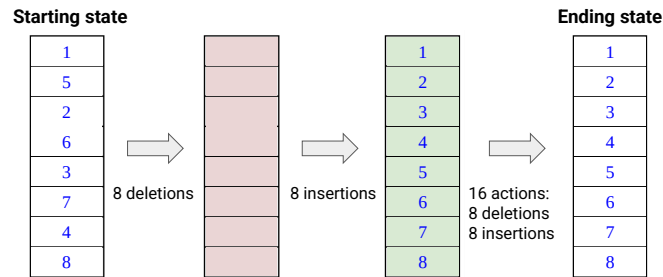
In NumberListAdapter.kt:

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): IntViewHolder{
    val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
        parent, false)
    val holder = IntViewHolder(layout)
    holder.row.setOnClickListener {
        // Do something on click
    }
    return holder
}
```

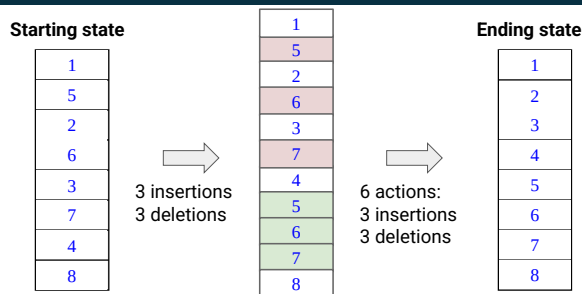
ListAdapter

- RecyclerView.Adapter
 - Disposes UI data on every update
 - Can be costly and wasteful
- ListAdapter
 - Computes the difference between what is currently shown and what needs to be shown
 - Changes are calculated on a background thread

Sort using RecyclerView.Adapter



Sort using ListAdapter



ListAdapter example

```
class NumberListAdapter: ListAdapter<Int,
    NumberListAdapter.IntViewHolder>(RowItemDiffCallback()) {

    class IntViewHolder(val row: View):RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }

    ...
}
```

DiffUtil.ItemCallback

Determines the transformations needed to translate one list into another

- areContentsTheSame(oldItem: T, newItem: T): Boolean
- areItemsTheSame(oldItem: T, newItem: T): Boolean

DiffUtil.ItemCallback example

```
class RowItemDiffCallback : DiffUtil.ItemCallback<Int>() {

    override fun areItemsTheSame(oldItem: Int, newItem: Int): Boolean {
        return oldItem == newItem
    }

    override fun areContentsTheSame(oldItem: Int, newItem: Int): Boolean {
        return oldItem == newItem
    }
}
```

Advanced binding

ViewHolders and data binding

```
class IntViewHolder private constructor(val binding: ItemViewBinding):
    RecyclerView.ViewHolder(binding.root) {

    companion object {
        fun from(parent: ViewGroup): IntViewHolder {
            val inflater = LayoutInflater.from(parent.context)
            val binding = ItemViewBinding.inflate(inflater,
                parent, false)
            return IntViewHolder(binding)
        }
    }
}
```

Using the ViewHolder in a ListAdapter

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
    IntViewHolder {  
        return IntViewHolder.from(parent)  
    }  
  
override fun onBindViewHolder(holder: NumberListAdapter.IntViewHolder,  
    position: Int) {  
    holder.binding.num = getItem(position)  
}
```

Binding adapters

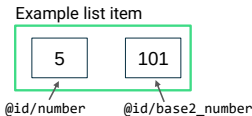
Let you map a function to an attribute in your XML

- Override existing framework behavior:
android:text = "foo" → `TextView.setText("foo")` is called
- Create your own custom attributes:
app:base2Number = "5" → `TextView.setBase2Number("5")` is called

Custom attribute

Add another `TextView` in the list item layout that uses a custom attribute:

```
<TextView  
    android:id="@+id/base2_number"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:textSize="24sp"  
    app:base2Number="@{num}" />
```



Add a binding adapter

Declare binding adapter:

```
@BindingAdapter("base2Number")  
fun TextView.setBase2Number(item: Int) {  
    text = Integer.toBinaryString(item)  
}
```

In `NumberListAdapter.kt`:

```
override fun onBindViewHolder(holder: NumberListAdapter.IntViewHolder,  
    position: Int) {  
    holder.binding.num = getItem(position)  
    holder.binding.executePendingBindings()  
}
```

Updated RecyclerViewDemo app

0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100

Multiple item view types

Add a new item view type

1. Create a new list item layout XML file.
2. Modify underlying adapter to hold the new type.
3. Override `getItemViewType` in adapter.
4. Create a new `ViewHolder` class.
5. Add conditional code in `onCreateViewHolder` and `onBindViewHolder` to handle the new type.

Declare new color item layout

```
<layout ...>  
    <data>  
        <variable  
            name="color"  
            type="android.graphics.Color" />  
        </data>  
    <androidx.constraintlayout.widget.ConstraintLayout ...>  
        <TextView  
            ...  
            android:backgroundColor="@{color.toArgb()}" />  
        <TextView  
            ...  
            android:text="@{color.toString()}" />  
    </androidx.constraintlayout.widget.ConstraintLayout>  
</layout>
```

New view type

- Adapter should know about two item view types:
 - Item that displays a number
 - Item that displays a color
- ```
enum class ITEM_VIEW_TYPE { NUMBER, COLOR }
```
- Modify `getItemViewType()` to return the appropriate type (as `Int`):  
`override fun getItemViewType(position: Int): Int`

## Override getItemViewType

```
In NumberListAdapter.kt:
override fun getItemViewType(position: Int): Int {
 return when(getItem(position)) {
 is Int -> ITEM_VIEW_TYPE.NUMBER.ordinal
 else -> ITEM_VIEW_TYPE.COLOR.ordinal
 }
}
```

## Define new ViewHolder

```
class ColorViewHolder private constructor(val binding: ColorItemViewBinding):
 RecyclerView.ViewHolder(binding.root) {

 companion object {
 fun from(parent: ViewGroup): ColorViewHolder {
 val inflater = LayoutInflater.from(parent.context)
 val binding = ColorItemViewBinding.inflate(inflater,
 parent, false)
 return ColorViewHolder(binding)
 }
 }
}
```

## Update onCreateViewHolder()

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
 RecyclerView.ViewHolder {

 return when(viewType) {
 ITEM_VIEW_TYPE.NUMBER.ordinal -> IntViewHolder.from(parent)
 else -> ColorViewHolder.from(parent)
 }
}
```

## Update onBindViewHolder()

```
override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position: Int) {
 when (holder) {
 is IntViewHolder -> {
 holder.binding.num = getItem(position) as Int
 holder.binding.executePendingBindings()
 }
 is ColorViewHolder -> {
 holder.binding.color = getItem(position) as Color
 holder.binding.executePendingBindings()
 }
 }
}
```

## Headers

## Headers Example

| Entrees  |        |
|----------|--------|
| Burger   | \$5.00 |
| Salad    | \$3.00 |
| Sandwich | \$4.00 |
| Drinks   |        |
| Coffee   | \$2.00 |
| Soda     | \$1.00 |

### 2 item view types:

- header item

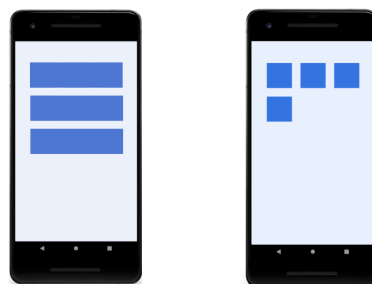
Drinks

- food menu item

Coffee \$2.00

## Grid layout

## List versus grid



## Specifying a LayoutManager

In MainActivity onCreate(), once you have a reference to the RecyclerView

- Display a list with LinearLayoutManager:  
`recyclerView.layoutManager = LinearLayoutManager(this)`
- Display a grid with GridLayoutManager:  
`recyclerView.layoutManager = GridLayoutManager(this, 2)`
- Use a different layout manager (or create your own)

- Arranges items in a grid as a table of rows and columns.
- Orientation can be vertically or horizontally scrollable.
- By default, each item occupies 1 span.
- You can vary the number of spans an item takes up (span size).

Create `SpanSizeLookup` instance and override `getSpanSize(position)`:

```
val manager = GridLayoutManager(this, 2)
manager.spanSizeLookup = object : GridLayoutManager.SpanSizeLookup() {
 override fun getSpanSize(position: Int): Int {
 return when (position) {
 0,1,2 -> 2
 else -> 1
 }
 }
}
```

## Summary

## Summary

In Lesson 10, you learned how to:

- Use `ListAdapter` to make `RecyclerView` more efficient at updating lists
- Create a binding adapter with custom logic to set View values from an XML attribute
- Handle multiple `ViewHolders` in the same `RecyclerView` to show multiple item types
- Use `GridLayoutManager` to display items as a grid
- Specify span size for an item in a grid with `SpanSizeLookup`

## Learn More

- [Create a List with RecyclerView](#)
- [RecyclerView](#)
- [ListAdapter](#)
- [Binding adapters](#)
- [GridLayoutManager](#)
- [DiffUtil](#) and [ItemCallback](#)

## Pathway

Practice what you've learned by completing the pathway:

[Lesson 10: Advanced RecyclerView use cases](#)





## Lesson 11: Connect to the internet

## About this lesson

### Lesson 11: Connect to the internet

- [Android permissions](#)
- [Connect to, and use, network resources](#)
- [Connect to a web service](#)
- [Display images](#)
- [Summary](#)

# Android permissions

## Permissions

- Protect the privacy of an Android user
- Declared with the `<uses-permission>` tag in the `AndroidManifest.xml`

## Permissions granted to your app

- Permissions can be granted during installation or runtime, depending on protection level.
- Each permission has a protection level: normal, signature, or dangerous.
- For permissions granted during runtime, prompt users to explicitly grant or deny access to your app.

## Permission protection levels

| Protection Level | Granted when? | Must prompt before use? | Examples                                        |
|------------------|---------------|-------------------------|-------------------------------------------------|
| Normal           | Install time  | No                      | ACCESS_WIFI_STATE, BLUETOOTH, VIBRATE, INTERNET |
| Signature        | Install time  | No                      | N/A                                             |
| Dangerous        | Runtime       | Yes                     | GET_ACCOUNTS, CAMERA, CALL_PHONE                |

## Add permissions to the manifest

In `AndroidManifest.xml`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
 package="com.example.sampleapp">
 <uses-permission android:name="android.permission.USE_BIOMETRIC" />
 <application>
 <activity
 android:name=".MainActivity" ... >
 ...
 </activity>
 </application>
</manifest>
```

## Internet access permissions

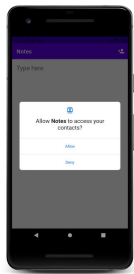
In `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

## Request dangerous permissions

- Prompt the user to grant the permission when they try to access functionality that requires a dangerous permission.
- Explain to the user why the permission is needed.
- Fall back gracefully if the user denies the permission (app should still function).

## Prompt for dangerous permission



## App permissions best practices

- Only use the permissions necessary for your app to work.
- Pay attention to permissions required by libraries.
- Be transparent.
- Make system accesses explicit.

## Connect to, and use, network resources

## Retrofit

- Networking library that turns your HTTP API into a Kotlin and Java interface
- Enables processing of requests and responses into objects for use by your apps
  - Provides base support for parsing common response types, such as XML and JSON
  - Can be extended to support other response types

## Why use Retrofit?

- Builds on industry standard libraries, like OkHttp, that provide:
  - HTTP/2 support
  - Connection pooling
  - Response caching and enhanced security
- Frees the developer from the scaffolding setup needed to run a request

## Add Gradle dependencies

```
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-moshi:2.9.0"

implementation "com.squareup.moshi:moshi:$moshi_version"
implementation "com.squareup.moshi:moshi-kotlin:$moshi_version"
kapt "com.squareup.moshi:moshi-kotlin-codegen:$moshi_version"
```

## Connect to a web service

## HTTP methods

- GET
- POST
- PUT
- DELETE

## Example web service API

URL	DESCRIPTION	METHOD
example.com/posts	Get a list of all posts	GET
example.com/posts/username	Get a list of posts by user	GET
example.com/posts/search?filter=queryterm	Search posts using a filter	GET
example.com/posts/new	Create a new post	POST

## Define a Retrofit service

```
interface SimpleService {
 @GET("posts")
 suspend fun listPosts(): List<Post>

 @GET("posts/{userId}")
 suspend fun listByUser(@Path("userId") userId: String): List<Post>

 @GET("posts/search") // becomes post/search?filter=query
 suspend fun search(@Query("filter") search: String): List<Post>

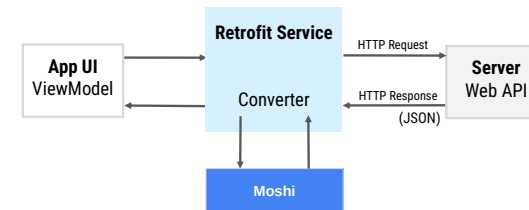
 @POST("posts/new")
 suspend fun create(@Body post : Post): Post
}
```

## Create a Retrofit object for network access

```
val retrofit = Retrofit.Builder()
 .baseUrl("https://example.com")
 .addConverterFactory(...)
 .build()

val service = retrofit.create(SimpleService::class.java)
```

## End-to-end diagram



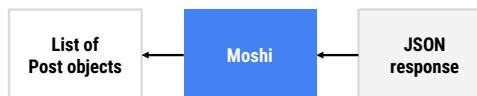
## Converter.Factory

Helps convert from a response type into class objects

- JSON (Gson or Moshi)
- XML (Jackson, SimpleXML, JAXB)
- Protocol buffers
- Scalars (primitives, boxed, and Strings)

## Moshi

- JSON library for parsing JSON into objects and back
- Add Moshi library dependencies to your app's Gradle file.
- Configure your Moshi builder to use with Retrofit.



## Moshi JSON encoding

```
@JsonClass(generateAdapter = true)
data class Post (
 val title: String,
 val description: String,
 val url: String,
 val updated: String,
 val thumbnail: String,
 val closedCaptions: String?)
```

## JSON code

```
{
 "title": "Android Jetpack: EmojiCompat",
 "description": "Android Jetpack: EmojiCompat",
 "url": "https://www.youtube.com/watch?v=sYGKUtM2ga8",
 "updated": "2018-06-07T17:09:43+00:00",
 "thumbnail": "https://i4.ytimg.com/vi/sYGKUtM2ga8/hqdefault.jpg"
}
```

## Set up Retrofit and Moshi

```
private val moshi = Moshi.Builder()
 .add(KotlinJsonAdapterFactory())
 .build()

val retrofit = Retrofit.Builder()
 .addConverterFactory(MoshiConverterFactory.create(moshi))
 .baseUrl(BASE_URL)
 .build()

object API {
 val retrofitService : SimpleService by lazy {
 retrofit.create(SimpleService::class.java)
 }
}
```

## Use Retrofit with coroutines

Launch a new coroutine in the view model:

```
viewModelScope.launch {
 Log.d("posts", API.retrofitService.searchPosts("query"))
}
```

# Display images

## Glide

## Add Gradle dependency

- Third-party image-loading library in Android
- Focused on performance for smoother scrolling
- Supports images, video stills, and animated GIFs

```
implementation "com.github.bumptech.glide:glide:$glide_version"
```

## Load an image

```
Glide.with(fragment)
 .load(url)
 .into(imageView);
```

## Customize a request with RequestOptions

- Apply a crop to an image
- Apply transitions
- Set options for placeholder image or error image
- Set caching policies

## RequestOptions example

```
@BindingAdapter("imageUrl")
fun bindImage(imgView: ImageView, imgUrl: String?) {
 imgUrl?.let {
 val imgUri = imgUrl.toUri().buildUpon().scheme("https").build()

 Glide.with(imgView)
 .load(imgUri)
 .apply(RequestOptions()
 .placeholder(R.drawable.loading_animation)
 .error(R.drawable.ic_broken_image))
 .into(imgView)
 }
}
```

# Summary

## Summary

In Lesson 11, you learned how to:

- Declare permissions your app needs in `AndroidManifest.xml`
- Use the three protection levels for permissions: normal, signature, and dangerous (prompt the user at runtime for dangerous permissions)
- Use the Retrofit library to make web service API calls from your app
- Use the Moshi library to parse JSON response into class objects
- Load and display images from the internet using the `Glide` library

## Learn More

- [App permissions best practices](#)
- [Retrofit](#)
- [Moshi](#)
- [Glide](#)

# Pathway

Practice what you've learned by completing the pathway:

[Lesson 11: Connect to the internet](#)

